

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Performance Analysis of Incremental Learning Approaches in Network Intrusion Detection

NGUYỄN QUANG MINH

minh.nq194444@sis.hust.edu.vn

Major: Data Science

Specialization: Data Science And Artificial Intelligence

Supervisor: Dr. Trần Hải Anh

Signature

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 07/2024

ACKNOWLEDGMENT

I would like to express my sincere gratitude to everyone who supported me throughout the completion of this thesis. I am particularly thankful to Dr Tran Hai Anh for his invaluable advice, guidance, and numerous other contributions during my research. I am thankful to my family and girlfriend for their support. I am also deeply grateful to all my friends for their care and encouragement. Additionally, I want to thank Zhou Da-Wei for providing his open-source tool on Class Incremental Learning and for his constructive feedback.

ABSTRACT

This thesis advances Network Intrusion Detection Systems (NIDS) by integrating Class Incremental Learning (CIL) to address the challenges of evolving cyber threats. Traditional NIDS have played a crucial role in safeguarding networks against malicious activities. However, they struggle to adapt to new attack patterns due to their static learning models. By incorporating CIL, NIDS can dynamically learn and adapt to novel attacks without forgetting previous knowledge.

This thesis evaluates several CIL methods, including Dynamic Expandable Representation (DER), Memory Efficient Expandable Model (MEMO), Learning without Forgetting (LWF), and Incremental Classifier and Representation Learning (ICARL), using benchmark datasets like KDD99, CIC-IDS-2017, ToN-IoT-Network, and UNSW-NB15. The results show that CIL-based NIDS significantly improve detection accuracy and adaptability compared to traditional models in continual learning setup. DER, in particular, demonstrated strong performance in maintaining high accuracy across incremental learning stages.

This thesis contributes to cyber-security by enhancing the adaptability and robustness of NIDS, providing a foundation for future work to further refine and deploy these techniques in real-world scenarios. The source code to reproduce the results is available at https://github.com/NguyenQuangMinh0504/CIL_NIDS

Student

(Signature and full name)

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Problem Statement.....	1
1.2 Background and Problems of Research	1
1.3 Research Objectives	2
1.4 Contributions	3
1.5 Organization of Thesis	3
CHAPTER 2. PRELIMINARIES AND LITERATURE REVIEW	5
2.1 Preliminaries	5
2.1.1 Class Incremental Learning.....	5
2.1.2 Deep Learning	8
2.2 Literature Review	9
2.2.1 Scope of Research.....	9
2.2.2 Related Works	9
CHAPTER 3. GENERIC IMPLEMENTATION SOLUTION AND BENCH- MARK METHODS.....	14
3.1 Generic Implementation Solution.....	14
3.1.1 Data Selection	14
3.1.2 Model Training	15
3.1.3 Model Evaluation.....	15
3.1.4 FC Layer Expanding.....	15
3.1.5 Exemplar Selection	15
3.2 Benchmark methods	15
3.2.1 LWF: Learning without Forgetting.....	15
3.2.2 ICARL: Incremental Classifier and Representation Learning	17

3.2.3 DER: Dynamic Expandable Representation.....	19
3.2.4 MEMO: Memory Efficient Expandable Model	20
3.2.5 Benchmark dataset	21
CHAPTER 4. NUMERICAL RESULTS.....	32
4.1 Evaluation Parameters.....	32
4.2 Data Preprocessing	32
4.2.1 Feature Encoding	32
4.2.2 Feature Processing	33
4.2.3 Dataset Processing	33
4.3 Implementation Detail.....	34
4.3.1 Network Architecture	34
4.3.2 Training Parameter setting	34
4.4 Comparison of Implemented Methods Among Different Dataset.....	35
4.5 Comparison of Implemented Methods With Different Exemplar Size	37
4.6 Experiment with Temperature parameter in Knowledge Distillation-based methods	40
4.7 Evaluation of DER with NME Classifier.....	42
CHAPTER 5. CONCLUSIONS	44
5.1 Summary	44
5.2 Suggestion for Future Works.....	44
REFERENCE	48
APPENDIX	50
A. Knowledge Distillation Loss Leads to Failed Coverage.....	50

LIST OF FIGURES

Figure 2.1	Class Incremental Learning Setup	5
Figure 2.2	Training with Exemplar Set	8
Figure 2.3	Training Generative Models Scheme	12
Figure 3.1	Training and evaluating model in CIL scenario	14
Figure 3.2	ICARL Exemplar Management compare to fixed number of Exemplar	18
Figure 3.3	DER Flow	20
Figure 3.4	MEMO Flow	21
Figure 3.5	t-SNE visualization of KDD99 dataset	29
Figure 3.6	t-SNE visualization of CIC-IDS-2017 dataset	30
Figure 3.7	t-SNE visualization of UNSW-NB15 dataset	30
Figure 3.8	t-SNE visualization of ToN-IoT-Network dataset	31
Figure 3.9	t-SNE visualization of Synthetic dataset	31
Figure 4.1	Model Architecture	34
Figure A.1	Failed Converge of LWF when trained on CIC-IDS-2017 dataset	50
Figure A.2	Pytorch Example of Failed Converge Training Instance . .	50

LIST OF TABLES

Table 3.1	Features set of KDD99 dataset	22
Table 3.2	KDD99's category and related attack type	22
Table 3.3	Number of instances per class of KDD99 dataset	23
Table 3.4	Features set of CIC-IDS-2017 dataset	25
Table 3.5	Number of instances per class of CIC-IDS-2017 dataset . .	26
Table 3.6	Number of instances per class of Ton-iot-network dataset . .	26
Table 3.7	Features set of Ton-Iot-Network dataset	27
Table 3.8	Features set of UNSW-NB15 dataset	28
Table 3.9	Number of instances per class of UNSW-NB15 dataset . . .	28
Table 4.1	Evaluation on KDD99 dataset	35
Table 4.2	Evaluation on CIC-IDS-2017 dataset	36
Table 4.3	Evaluation on ToN-IoT-Network dataset	36
Table 4.4	Evaluation on UNSW-NB15 dataset	36
Table 4.5	Evaluation on Synthetic dataset	36
Table 4.6	Evaluation on KDD99 dataset, with different exemplar size .	38
Table 4.8	Evaluation on Ton-IoT-Network dataset, with different exem- plar size	38
Table 4.7	Evaluation on CIC-IDS-2017 dataset, with different exem- plar size	39
Table 4.10	Evaluation on Synthetic dataset, with different exemplar size	39
Table 4.9	Evaluation on UNSW-NB15 dataset, with different exemplar size	40
Table 4.11	Evaluations of LWF and ICARL on Synthetic dataset with different temperature parameter	41
Table 4.12	Evaluations of LWF and ICARL on KDD99 dataset with different temperature parameter	41
Table 4.13	Evaluations of DER on multiple datasets using NME Classifier	43

LIST OF ABBREVIATIONS

Abbreviation	Definition
CIL	Class Incremental Learning
DER	Dynamic Expandable Representation
GAN	Generative Adversarial Network
ICARL	Incremental Classifier And Representation Learning
KD	Knowledge Distillation
LWF	Learning Without Forgetting
MEMO	Memory Efficient Expandable Model
NIDS	Network Intrusion Detection System
NME	Nearest Mean of Exemplar

CHAPTER 1. INTRODUCTION

1.1 Problem Statement

Network security has heavily relied on traditional Intrusion Detection Systems to identify and stop malicious activity [1]. However, their effectiveness suffers significantly as the threat landscape evolves at an alarming rate. New attack techniques and vulnerabilities emerge constantly, leaving traditional NIDS models struggling to adapt. This vulnerability comes from their reliance on pre-trained data, which cannot capture the ever-changing nature of cyber threats. My thesis investigates Class Incremental Learning in NIDS, a promising paradigm that enables detectors to continuously learn and adapt to novel attack patterns without the need for retraining on the entire dataset. By incorporating CIL techniques, NIDS can evolve alongside the threat landscape, maintaining robust detection capabilities in the face of unseen attacks.

1.2 Background and Problems of Research

Intrusion Detection System (IDS) are essential in cybersecurity for monitoring network traffic and identifying potential threats automatically at the network or host level in a timely manner. According to [2], IDS can be divided into Host-based Intrusion Detection System (HIDS) and Network Intrusion Detection System (NIDS). HIDS operates on individual host, monitoring system activities (system calls, file system modification, . . .), logs (system logs, applications logs, . . .), disk resources and other host specific activities to detect malicious actions. Although HIDS provides detailed monitoring and effective against local threats, it faces challenges with scalability and consumption of system resources. On the other hand, NIDS monitors network traffic by analyzing data packets at network point to identify malicious actions. NIDS use network traffic, collected via mirroring by network devices such as switches, routers to identify threats within network traffic [2]. It offers broad network visibility, centralized monitoring, and minimal impact on host performance. However, NIDS struggles with encrypted traffic, high a high rate of false positives, and lack insight into host-specific threats. While each system has its strengths, they are often used together for a more comprehensive security approach. An emerging trend is the integration of machine learning and deep learning techniques into both HIDS and NIDS, enhancing their ability to detect complex patterns and novel threats.

Traditional NIDS can be broadly categorized into two main approaches: Signature-based detection and Anomaly-based detection [3] [4]. Signature-based detection

relies on a pre-defined library of attack signatures, representing known malicious patterns. NIDS compares incoming packages against these signatures, flagging any matches as potential intrusions. While effective against well-documented attacks, this approach suffers from inherent limitations. Novel or zero-day attack which does not present in the signature library will evade detection, making the system vulnerable to the evolving threat landscape. Anomaly-based detection attempts to identify deviations from normal network behavior, assuming that malicious activity will manifest as anomalous patterns. Statistical model or machine learning model are trained on historical network data to establish baselines of normality. Deviations from these baselines raise suspicion and trigger alerts. However, anomaly-based detection can be prone to false positives, flagging benign activities as suspicious due to their inherent deviations from the norm. Despite their limitations, traditional NIDS remain valuable tools in network security domain. They offer real-time monitoring capabilities, providing early warnings of potential intrusions. However, their effectiveness hinges on the ability to keep pace with the dynamic threat landscape, a challenge that need the exploration of more advanced and adaptable approaches such as CIL, which forms the focus of this thesis.

1.3 Research Objectives

The research objectives of this thesis are as follow:

1. **Integration of Class Incremental Learning in Network Intrusion Detection:** Traditional NIDS find it difficult to keep up with the fast-changing landscape because they depend on pre-trained data, which does not account for new attack techniques. By integrating CIL, NIDS can evolve with these threats, ensuring they maintain strong detection capabilities.
2. **Comparative Analysis of CIL methods:** This thesis aim to conduct a detailed comparison of various CIL techniques. Specifically, it will focus on Dynamic Expandable Representation (DER), Learning Without Forgetting (LWF), Incremental Classifier and Representation Learning (ICARL), and Memory Efficient Expandable Model (MEMO). By evaluating the advantages and disadvantages of each model, this thesis seeks to identify the most suitable approach for implementation in NIDS. This comparative analysis will provide valuable insights for developing NIDS that can effectively adapt to evolving cyber threats without compromising performances on previous learned attack patterns.
3. **Evaluations on Multiple Datasets:** To ensure the CIL methods are robust and generalizable to real-world situations, their performance will be evaluated on a

range of benchmark datasets. This includes established datasets like KDD99, CIC-IDS-2017, and UNSW-NB15, ToN-IoT-Network and a synthetic dataset. By testing across such diverse data, the research aims to demonstrate that the findings are not limited to specific datasets and can be reliably applied to various real-world network intrusion scenarios.

1.4 Contributions

This thesis makes several contributions to the field of Network Intrusion Detection System and Class Incremental Learning:

- **Integration of Class Incremental Learning in NIDS:** This thesis suggests the application of CIL techniques to NIDS, addressing the challenge of continuously evolving cyber threats. By enabling NIDS to learn and adapt to new attack patterns without retraining on the entire dataset, this approach maintains capabilities against emerging threats.
- **Comparative Analysis of CIL methods:** This thesis provides an extensive comparative analysis of various CIL approaches, including DER, LWF, ICARL and MEMO.
- **Evaluations on Multiple Datasets:** The research evaluates the performance of proposed CIL methods across multiple benchmark datasets, including KDD99, CIC-IDS-2017, UNSW-NB15, ToN-IoT-Network and a synthetic dataset. This comprehensive evaluation ensures the robustness and generalizability of the finding.

1.5 Organization of Thesis

The remainder of my thesis is organized as follow:

Chapter 2. Preliminaries and Literature Review: This chapter provides a foundational understanding of key concepts and a comprehensive review of existing literature related to the research. The chapter is divided into two primary sections: Preliminaries and Literature Review. Section Preliminaries introduces and elaborates on essential concepts such as Class Incremental Learning and its significance in the context of Network Intrusion Detection Systems . This part covers the definition, motivations, and terminologies associated with CIL, alongside a discussion on deep learning terminologies necessary for the subsequent chapters. The second section, Literature Review, details contemporary methods for detecting malicious network activity through ML/DL techniques. It also investigates the latest approaches in the field of CIL. By analyzing the related work in both NIDS and CIL, this section establishes the motivation for undertaking this thesis research.

Chapter 3. Generic Implementation Solution and Benchmark Methods: This chapter explains the methods for implementing and evaluating CIL within NIDS. The chapter contains two sections: Generic implementation solution and Benchmark methods. Generic implementation solution section explores the flow of machine learning model in CIL scenario. Benchmark methods begins with a detailed description of several key methods, including DER, MEMO, LWF and ICARL. Next, it introduces the benchmark datasets employed in the study, such as KDD99, CIC-IDS-2017, ToN-IoT-Network, and UNSW-NB15, and a synthetic dataset. Visualizations using the t-SNE technique are provided to illustrate the structure and separability of these datasets.

Chapter 4. Numerical Results: In this chapter, I evaluate the performances the previously mentioned methods in the field NIDS. I evaluate the methods across multiple datasets, including KDD99, CIC-IDS-2017, ToN-IoT-Network, and UNSW-NB15. The chapter begins with an explanation of the metrics used in the experiments. It then covers the implementation aspects, covering dataset processing, model architecture, and training parameters. The chapter end with the analyzation of the performance of the implemented methods under different conditions.

Chapter 5. Conclusions: In this final chapter, the finding are summarized. The limitation and future work are also discussed.

CHAPTER 2. PRELIMINARIES AND LITERATURE REVIEW

2.1 Preliminaries

This section is divided into two parts. In the first part, I will discuss Class Incremental Learning, covering its definition, motivations, the rationale for its application in NIDS, and associated CIL terminologies. The second part will clarify deep learning terminologies, which are essential for understanding the architecture described in the subsequent sections.

2.1.1 Class Incremental Learning

a, Definition

Class Incremental Learning aims to learn from an evolutive stream with incoming new classes. Assume there is a sequence of B training tasks $\{D^1, D^2, \dots, D^B\}$ without overlapping classes, where $D^b = \{(x_i^b, y_i^b)\}_{i=1}^{n_b}$ is the b -th incremental step with n_b training instances. $x_i^b \in R^D$ is an instance of class $y_i \in Y_b$. Y_b is the label space of task b , where $Y^b \cap Y^{b'} = \emptyset$ for $b \neq b'$. We can only access data from D^b when training task b . The goal of CIL is to continually build a classification model for all classes. In other words, the model should not only acquire the knowledge from current task D^b but also preserve the knowledge from former tasks. After each task, the trained model is evaluated over all seen classes $\{Y_b = Y_1 \cup \dots \cup Y_b\}$.

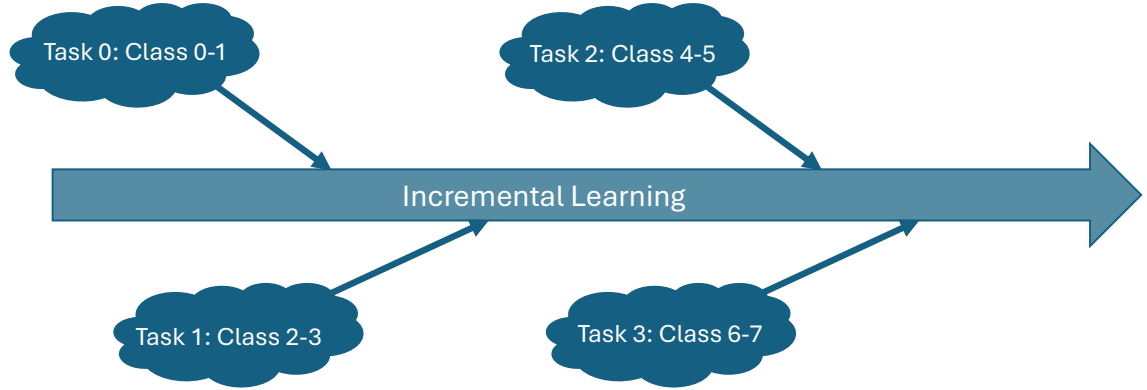


Figure 2.1: Class Incremental Learning Setup

b, Motivation

Class Incremental Learning is a methodology that evolves continuously updating a model's knowledge base by adding new classes or category over time. This approach is particularly useful in scenarios where data distribution is dynamic, and new classes need to be incorporated without retraining the model from scratch. Motivation for adopting class incremental learning techniques comes from several factors:

1. **Resource Efficiency:** Traditional retraining of models with new data can be computationally expensive, especially when dealing with large dataset or complex models. CIL allows for efficient update by only focusing on new classes or data, reducing the computation burden.
2. **Scalability:** As dataset grow continuously or new category emerge in real-world applications, scalability becomes a significant concern. CIL enables models to scale by incrementally incorporating new classes without revisit the entire train process.
3. **Adaptability to Dynamic Environments:** Many real world application operate in dynamic environments where the data distribution can change over time. CIL provides the flexibility to adapt to these changes by seamlessly integrating new knowledge without disrupting the model's performance.
4. **Continuous Learning:** CIL align with the concept of lifelong learning, where the models continuously improve their performance over time by learning from new experiences. This continuous learning paradigm is particularly advantageous in applications where staying up-to-date with the latest information is critical.

c, Motivation for CIL in NIDS

In the context of network intrusion detection, where the primary objective is to identify and respond to malicious activities or anomalous behavior in network traffic, the motivation for adopting class incremental learning techniques is due to the following reasons:

1. **Adaptation to Evolving Threats:** The landscape of cyber threats is constantly evolving, with new attack techniques and patterns emerging regularly. Class incremental learning enables intrusion detection systems to adapt to these evolving threats by incorporating new attack classes or patterns without requiring a complete retraining of the model. This adaptability is crucial for maintaining the effectiveness of intrusion detection systems in detecting novel and sophisticated attacks.
2. **Reduced Latency in Response:** In network intrusion detection, timely detection and response to threats are critical to mitigate potential damages. Class incremental learning enables intrusion detection system to update their knowledge base rapidly with new attack signatures or patterns, leading to reduced latency in detecting and responding to emerging threats. This rapid response capability enhances the effectiveness of intrusion detection systems in miti-

gating the impact of cyber attacks.

d, Related terminologies

Catastrophic forgetting

Catastrophic forgetting is a significant challenge in the field of machine learning, particularly in incremental learning system. It refers to the tendency of a neural network to completely previously learned information upon learning new data. This phenomenon is especially problematic in scenarios requiring continuous learning, such as in NIDS, where the model must adapt to new types of network intrusion without losing its ability to detect previous known threats. In a traditional learning setup, the model is trained on a fixed dataset, and any new learning involves retraining from scratch, which is inefficient and impractical for real-time applications. However, in incremental learning, where new data is continuously fed into the system, catastrophic forgetting can severely degrade the model's performance by overwriting the weights associated with old knowledge. Addressing catastrophic forgetting is crucial for developing robust and adaptive machine learning systems that can maintain their performance over time in dynamic environments.

Exemplar Set

In accordance with the definition provided in [a], the model's data accessibility within each incremental task is restricted to the dataset designed for that particular task, represented as D_b . This constraint serves a dual purpose: to uphold the integrity of user privacy and to mitigate the demands placed on storage resources. However, under certain circumstances, a degree of flexibility is permitted, allowing the model to retain a collection of exemplars – a set of selected instances representative of previous tasks, thereby allows knowledge preservation across tasks.

Exemplar set is an extra collection of instances from former tasks $\epsilon = \{(x_j, y_j)\}_{j=1}^M, y_j \in Y_{b-1}$. With the help of the exemplar set, the model can utilize $\epsilon \cup D^b$ for the update within each task. The model manages the exemplar set after the training process of each task.

Exemplar sets enhance classification performance by serving as reference points for previously learned classes, enabling the model to make more accurate predictions with both new and existing data. Nonetheless, using exemplar sets introduces challenges like memory limitations as the number of classes increases, and data imbalance during training with new classes. Researchers are investigating techniques for compressing exemplars or adopting more efficient storage methods. Conversely, some researchers are exploring alternative approaches for continual learning that do not depend on storing old data, known as exemplar-free learning.

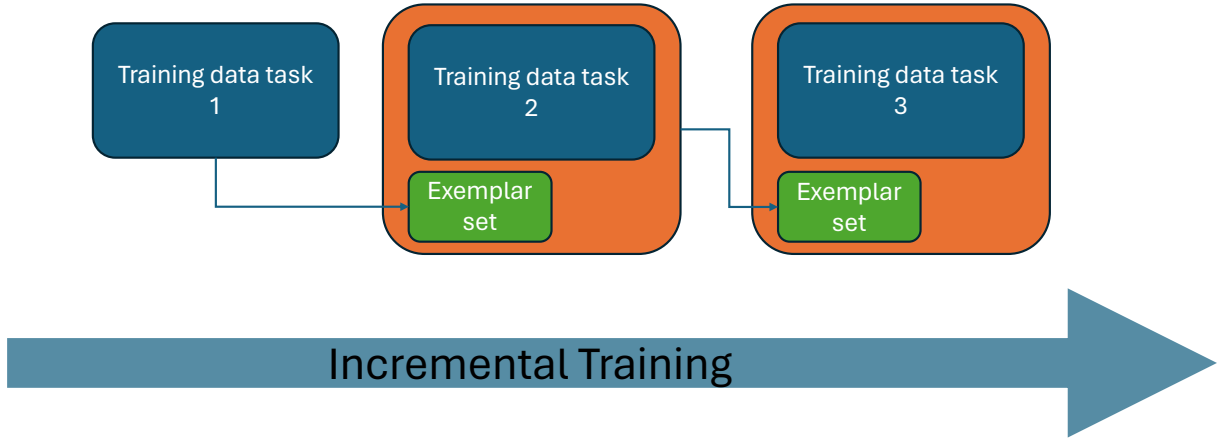


Figure 2.2: Training with Exemplar Set

2.1.2 Deep Learning

Feature extractor Φ : In deep learning models, a feature extractor is a component responsible for transforming raw input data into a more informative and compact representation. This representation, also known as features, captures the essential characteristics relevant to the specific task the model is trying to accomplish.

Fully Connected Layer $\mathcal{F}$: Take the extracted features from the previous layers (Feature Extractor) and performs classification, regression or other tasks. It does this by learning a mapping from the features to the output. In general, given an input x and a target y , a deep learning model can be defined as $\mathcal{F}(\Phi(x))$, where the target is to minimize the distance function $Distance(y, \mathcal{F}(\Phi(x)))$

Loss function $\mathcal{L}$: A loss function is serve as a vital tool for evaluating how well a model performs on a given task. By minimizing the overall loss, the model aims to improve its prediction and reduce the discrepancy between its outputs and the desired outcomes. Commonly employed in classification tasks, cross entropy loss measures the difference between probability distribution of the model's prediction and the true probability distribution of the desired outcome. The formula of cross entropy loss is:

$$\mathcal{L} = -y * \text{Log}(\mathcal{S}(\mathcal{F}(\Phi(x)))) \quad (2.1)$$

where y is the ground truth probability of the instance, and $\mathcal{S}$ is the Softmax function:

$$\mathcal{S}(z_i) = \frac{\exp(z_i)}{\sum \exp(z_j)} \quad (2.2)$$

Expanding the FC layer: As the number of classes increases with the introduc-

tion of new tasks, the size of the FC layer must be adjusted accordingly. Let denote the layer from former task as $\mathcal{F}_{t-1}$, the current FC layer $\mathcal{F}_t$ is created by adding new weights $\mathcal{F}'_t$ to $\mathcal{F}_{t-1}$. The size of $\mathcal{F}'_t$ corresponds to the number of classes in the new task, while the size of $\mathcal{F}_{t-1}$ represents the number of classes the model has previously learned. Therefore, the FC layer of the current task can be defined as:

$$\mathcal{F}_t = \mathcal{F}_{t-1} + \mathcal{F}'_t \quad (2.3)$$

2.2 Literature Review

2.2.1 Scope of Research

The primary objective of this research is to conduct a comprehensive analysis of various CIL methods in the context of NIDS. The thesis aims to explore the effectiveness of CIL approaches such as DER, LWF, ICARL, and MEMO. By evaluating these methods across multiple benchmark datasets, including KDD99, CIC-IDS-2017, UNSW-NB15, ToN-IoT-Network, and a synthetic dataset, the study seeks to establish a robust framework for enhancing the adaptability and accuracy of NIDS. The outcomes of this analysis are expected to contribute to the development of more resilient and efficient NIDS, capable of evolving with the introduction of new types of network intrusions without requiring complete retraining from scratch. This research excludes traditional machine learning techniques and optimizations for specific hardware. While the research offers a deep dive into CIL methodologies, it doesn't explore how these methods would integrate with existing network infrastructure or how performance might vary on different hardware platforms. Additionally, it doesn't consider the human aspect of NIDS deployment, such as user training and how human behavior might affect the system's effectiveness. The evaluation happens in a controlled environment, so real-world performance and the impact of deploying the system in real-time aren't analyzed. Finally, the research avoids getting into the details of specific attack techniques or creating new signatures for attacks.

In summary, by keeping the focus narrow on algorithmic and methodological advancements, the research prioritizes a clear evaluation of how well CIL approaches work in a controlled experimental setting.

2.2.2 Related Works

This section reviews the existing research and methodologies in the domain of NIDS and CIL. This review provide a comprehensive background for the current study within the the broader landscape of NIDS and CIL, identifying opportunities

for further advancements.

a, Network Intrusion Detection System

Machine learning and deep learning methods are being widely used in NIDS research. These techniques can analyze vast amounts of data and identify complex patterns that might be missed by traditional methods. Below are the mentions:

Machine Learning:

1. **Decision Tree:** Decision Tree is a hierarchical structure resembling a tree, where internal nodes represent features, branches represent decision rules and leaf nodes are the predictions. Decision Tree automatically chooses the optimal features to construct a tree and then prunes unnecessary branches to prevent over-fitting. Numerous advanced learning algorithms, such as Random Forest and XGBoost, are composed of multiple decision trees. Shone et al. [5] experimented with combining Random Forest and a dimension reduction technique on the KDD99 and NSL-KDD datasets.
2. **K-Nearest Neighbors:** K-Nearest Neighbors calculates the nearest data points of the query data based on the chosen distance function, and it assigns the most prevalent class to the data. [2] evaluate the performance of K-Nearest Neighbors against other machine learning and deep learning methods on dataset KDD99, UNSW-NB15 and CIC-IDS-2017.
3. **Support Vector Machine (SVM):** Support Vector Machine is a supervised machine learning algorithm based on the idea of max-margining separation hyper-plane in n-dimensional feature space. In case the data is not linear separable, a kernel function is used to project data into higher dimension space for separation. [6] evaluated linear SVM and non linear SVM using NIDS dataset.

Deep Learning:

1. **Auto Encoder:** Auto Encoder is a type of network used in unsupervised learning tasks. Its primary purpose is to learn efficient encodings of input data by compressing it into a latent-space representation and then reconstructing the output from this representation. The process ensures that the output is as close to the original input as possible. Auto Encoder usually contains 2 parts: Encoder (compresses the input data into latent space) and Decoder (reconstructs the input data from the compressed representation in the latent space). Some works propose to use the Encoder part of Auto Encoder as feature extractor and dimension reduction. [7] use Encoder part of Auto Encoder for features extractor and non linear SVM as classifier. Their paper shows better classifica-

tion results with NSL-KDD dataset. The training time also decrease compared to using non linear SVM only on raw data.

2. **Convolutional Neural Network:** Convolutional Neural Network (CNN) is a class of deep learning architecture famous for their efficiency in tasks evolving spatially structured data (pattern recognition, image classification, ...). The core component of CNN is convolutional layer, which employ filters that convolve over input data. This enables CNN to capture spatial relationship and translational invariance - the ability of recognizing pattern regardless of their location in the input space. [8] use CNN together with feature reduction techniques on KDD99 dataset and compare the result with machine learning approaches. They got better accuracy compared to ML approach, but the accuracy for minor classes was still very low.

b, Class Incremental Learning

CIL aims to enable models to learn new classes over time while retaining knowledge of previously learned classes. This section reviews various methods and approaches in CIL, highlighting their unique strategies and contributions to the field.

Exemplar-Based Methods: Exemplar-Based Methods involve maintaining a small set of representative instances from previous task to mitigate the problem of catastrophic forgetting. Given the limited number of exemplars, some methods suggest selecting the most informative ones. ICARL recommends using a herding strategy, selecting exemplars closest to the mean feature vector of each class. Other strategies include Plane Distance Based Sampling, which picks samples nearer to the decision boundary as they are more representative than those farther away, or entropy-based sampling, where samples with higher entropy are more likely to be chosen. Chaudhry et al. [9] compare the results of multiple exemplar-based methods, such as herding, Plane Distance Based Sampling, and entropy-based sampling, showing that herding was the most effective. Bang et al. [10] argue that exemplars should not only represent their class but also be discriminative against other classes. They propose using a variety of exemplars, including the most representative (close to the center of distribution) and the most discriminative (near the decision boundary).

Generative Replay Methods: Generative replay methods uses generative models to create synthetic data that resemble previously learned classes. This synthetic data is then used to train the model alongside new class data. The paper "Continual Learning with Deep Generative Replay" by Shin et al. [11] utilizes generative models to create pseudo-data for previously learned classes. The generator for a

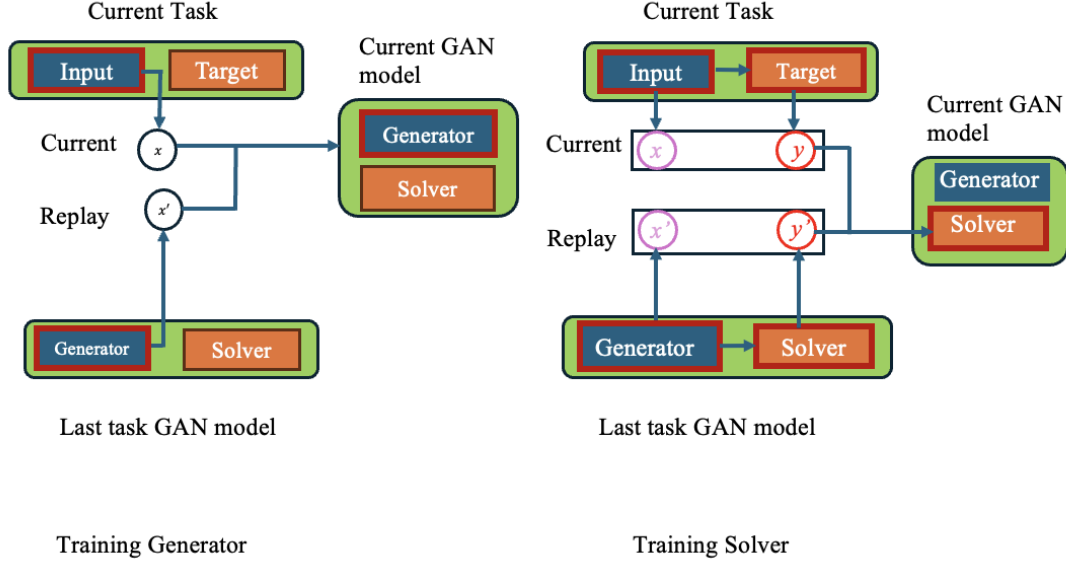


Figure 2.3: Training Generative Models Scheme

new task is trained using the data generated by the previous generator along with the data from the new task, as shown in figure 2.3. The classifier is trained on a combination of pseudo-inputs and targets produced by the previous task's generator and classifier, as well as the inputs and targets from the new task. This approach effectively mimics the rehearsal process without requiring the storage of real data. However, the training of the new generator relies on data produced by the old generator. This process may experience information loss during knowledge transfer and is prone to accumulating errors. To address this issue, the paper "Exemplar-Supported Generative Reproduction for Class Incremental Learning" [12] suggests employing a separate generator for each task. Additionally, the paper recommends incorporating exemplars to enhance model accuracy. Nevertheless, using a new generator for each task results in a linear increase in memory usage, which may be impractical in real-world applications.

Network Expanding Methods: Network expanding methods dynamically adjust the network's architecture by adding new component to accommodate new tasks while preserving previously learned knowledge. DER model from paper "DER: Dynamically Expandable Representation for Class Incremental Learning" by Yan et. al. [13] is a notable example. DER expands a new backbone for each incremental task and aggregates features from both old and new backbones. This approach freezes the old backbone to prevent overwriting and forgetting of previously learned features. DER's structure ensures that features learned for earlier

tasks remain intact while new backbones learn to detect new data, thus mitigating the forgetting issue. DER offer several advantages, including the ability to retain previous knowledge and adapt to new tasks without extensive retraining. However, it also face challenges such as increased computational and memory costs due to the growing network size. Solutions like model compression and efficient memory management are being explored to address these issues. Wang et al. propose incorporating an additional compression stage using knowledge distillation in their paper [14]. This technique allows the model to maintain only a single backbone, thereby reducing computational costs. However, this approach eliminates the frozen weights in the compressed, which can result in the forgetting of features from previous classes. MEMO (Memory Efficient Expandable Model) by Zhou et al. [15] adopts a different strategy by separating the network structure into generalized blocks (shallow layers) and specialized blocks (deep layers), expanding only the specialized blocks required for new tasks. This method optimizes memory usage while preserving previously learned information.

Regularization-Based Methods: Regularization-based methods introduce constraints during the training process to prevent the model from significantly altering the weights associated with previously learned classes. EWC [16] is the first to applied. EWC applied a penalty to changes in important weights for previously learned classes. The important of each weight is based on how crucial it was for previous task, and significant changes to these weights are penalized to reduce forgetting.

CHAPTER 3. GENERIC IMPLEMENTATION SOLUTION AND BENCHMARK METHODS

This chapter explains the methods for implementing and evaluating CIL within NIDS. The chapter is divided into two sections: Generic implementation solution and Benchmark methods. Generic implementation solution section explain the process of training and evaluating machine learning model in CIL scenario. Benchmark methods section provides an in-depth discussion on various benchmark methods used to evaluate the performance of CIL models, including LWF, ICARL, DER, and MEMO. Furthermore, it introduces the benchmark datasets employed in the study, such as KDD99, CIC-IDS-2017, TON-IoT-Network, and UNSW-NB15, and explains the process of generating a synthetic dataset to evaluate model performance. Visualizations using the t-SNE technique are provided to illustrate the structure and separability of these datasets.

3.1 Generic Implementation Solution

This section explains the process involved in developing and evaluating a machine learning model in a CIL scenario. The process comprises five phases: Data Selection, Model Training, Model Evaluation, FC Layer Expanding, and Exemplar Selection, as illustrated in Figure 3.1. The process is iterative, repeating whenever the model encounters a new task.

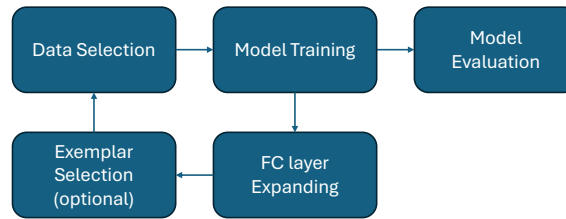


Figure 3.1: Training and evaluating model in CIL scenario

3.1.1 Data Selection

In a typical deep learning workflow, the dataset is split into a training set and a test set. The model is trained on the training set and evaluated on the test set. However, in a class incremental learning scenario, the dataset is divided into smaller non-overlapping subsets, with no shared classes between them, each representing a distinct task. Each subset is then further split into a training set and a test set. For each task, the model is trained on that task's training set and evaluated on the test set from the current task as well as all previous tasks. This approach allows the model to be assessed in a continual learning manner.

This phase selects the training and testing data for the task the model is currently facing. The training data for a task might contain examples from earlier classes.

3.1.2 Model Training

The model training utilizes a neural network architecture. Various studies propose different methods to address catastrophic forgetting. The training process involves standard techniques such as using back propagation to update weights, along with strategies specific to CIL. In my benchmarked methods, the CIL strategy includes backbone expansion or knowledge distillation.

3.1.3 Model Evaluation

The model is evaluated on the test set, which includes test data from all tasks, using CIL-relevant metrics like Last Accuracy and Average Accuracy. These metrics assess the model's effectiveness in detecting network intrusions and its capability to mitigate catastrophic forgetting.

3.1.4 FC Layer Expanding

In traditional setups, where the number of classes remains constant, the FC layer is fixed in size accordingly. In CIL, the number of classes grows as new tasks are introduced. The FC layer's output size equals the number of classes the model has learned. As the model faces new tasks, the FC layer must expand to accommodate additional classes.

3.1.5 Exemplar Selection

Exemplar selection is employed for models that utilize exemplars in their configuration. This phase selects exemplars based on specific criteria. Various studies propose different methods for choosing exemplars, with some suggesting the use of GAN models to generate additional training data. In my benchmarked methods, LWF does not use exemplars, whereas ICARL, DER, and MEMO do. The exemplar selection method used is the herding strategy, as suggested by ICARL.

3.2 Benchmark methods

3.2.1 LWF: Learning without Forgetting

A research paper by Z Li et al. [17] titled "Learning without Forgetting" addresses the challenge of forgetting old information when learning new things. Their approach uses a technique called Knowledge Distillation. When facing new tasks, the system first saves a copy of current existing model to keep current knowledge. Then the original model is trained with the data of new classes. To ensure the original model retains knowledge from the old tasks, knowledge distillation loss is used in the training phrase. Distillation loss involves using the predictions from the

saved model to guide the original model, helping it remember what it previously learned.

Knowledge distillation loss, introduced by Hinton et al. [18] in paper "Distilling the knowledge in a neural network" addresses the challenge of deploying powerful machine learning models. The traditional approaches of ensemble learning, where multiple models are trained and their predictions averaged, often lead to superior performance. However, this method becomes cumbersome and computationally expensive, especially for large models when deployed for real world applications. Knowledge distillation loss offers a solution by enabling the transfer of knowledge from complex ensembles (or large, regularized models) to a single, smaller model. This facilitates easier deployment while maintaining good performance.

Knowledge distillation loss comprises of two losses : The first loss is the knowledge distillation loss. This loss encourages the student (the smaller model trying to learn the underlying patterns and relationship within the data that the original model has captured) to mimic the teacher's (ensembled model) performance. The authors propose incorporating a temperature parameter T greater than 1 into the softmax function (2.2). This softens the probability distribution of the teacher's output, making them less peaked and more informative for the student (3.1). It provides richer information per training example compared to hard labels and lead to smoother gradients during training, thus allow the student to learn effectively even with less data than the teacher. The knowledge distillation loss is the soft cross entropy loss between the student's predictions with the teacher's predictions. The second loss function is the normal cross entropy loss between the student's predictions with the correct labels, as in (2.1)

$$\mathcal{S}_{soft}(z_i) = \frac{\exp(z_i/T)}{\sum \exp(z_j/T)} \quad (3.1)$$

The authors of LWF paper advance the idea of knowledge distillation in the context of class incremental learning. Whenever the model encounters new classes to learn (a new task), a frozen copy of the current model is stored. This frozen model acts as the teacher that hold the knowledge from previous task. This frozen model acts as the teacher that holds the knowledge from previous tasks. Meanwhile, the original model, now considered as the student, continues training on the data for the new classes. This approach helps the student retain valuable information from the past tasks while simultaneously adapting to the new data. Given an connection

x , training task t , the formula of knowledge distillation loss is as followed:

$$\mathcal{L}_{old} = -\mathcal{S}_{soft} \left(-\overline{\mathcal{F}_{t-1}} \left(\overline{\Phi_{t-1}}(x) \right) \right) * \text{Log} \left(\mathcal{S}_{soft} \left(\mathcal{F}_{t-1} \left(\Phi_{t-1}(x) \right) \right) \right) \quad (3.2)$$

For learning new task, the model using the normal cross entropy between the ground truth label and the output $\mathcal{F}'_t$ of the current model.

$$\mathcal{L}_{new} = -y * \text{Log} \left(\mathcal{S} \left(\mathcal{F}'_t \left(\Phi_{t-1}(x) \right) \right) \right) \quad (3.3)$$

The final loss of the model is:

$$\mathcal{L} = \mathcal{L}_{old} + \mathcal{L}_{new} \quad (3.4)$$

3.2.2 ICARL: Incremental Classifier and Representation Learning

The paper ICARL: Incremental Classifier and Representation Learning by Rebuffi et al. [19] extends the work of LWF in the field of Class Incremental Learning. The authors demonstrate that incorporating a small amount of past data, referred to as exemplars, significantly enhances the model's performance. ICARL proposes a novel classifier that combines three key elements: employing a herding strategy for exemplar selection, classifying based on nearest-mean-of-exemplar, and utilizing knowledge distillation loss for representation learning.

a, Exemplar set management

Continuously learning from a data stream necessitates managing the size of the exemplar set. A common approach is to maintain a fixed number of exemplars per class. However, this leads to a linear increasing memory size as the data stream evolves. This linear growth becomes impractical for real-world learning system with limited memory. To address this issue, ICARL proposes an alternative strategy: storing a fixed number of class M , as demonstrated in figure 3.2 Here, the model dynamic allocates storage based on the number of classes encounter Y_b . The model keeps $\lfloor M/Y_b \rfloor$ instances per class, where $\lfloor . \rfloor$ denotes the floor function. This approach effectively maintains a constant memory size for the exemplar set, mitigating storage concerns as the system encounters new classes.

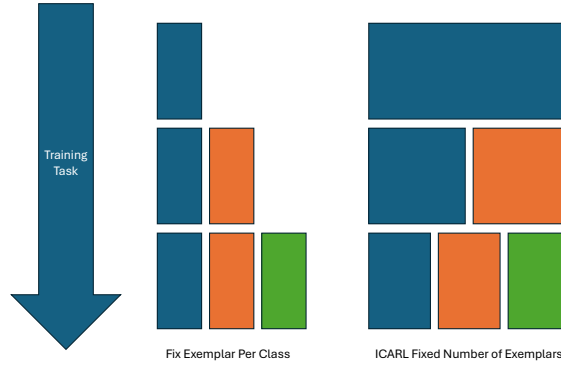


Figure 3.2: ICARL Exemplar Management compare to fixed number of Exemplar

b, Exemplar Selection

In incremental learning, exemplars set are representative instances of each known class, which is selected from the entire training set. A simple approach might involve randomly picking exemplars for each class, ensuring a diverse set. However, with limited storage for exemplars, ICARL proposes a more strategic approach called herding. This method prioritizes selecting the most representative data points for each class, maximizing the information content stored within the limited exemplar set. Given the instance set $X = [x_1, x_2, \dots, x_n]$ from class y , herding first calculates the class center with current embedding. Afterward, it calculates and rank the distance of each instance to the class center in ascending order. The exemplars are then selected based on ranking, e.g., the top $[M/y_b]$ instances with the least distance. Since the class center can be seen as the most representative pattern of each class, selecting exemplars near the class center also enhance the representativeness of exemplars.

c, Nearest-Mean-of-Exemplars Classification

Traditionally, deep learning models predict class labels by finding the class with the highest output score after processing the input data: $y = \operatorname{argmax}(\mathcal{F}(\Phi(x)))$. ICARL, however, introduces a different approach called nearest-mean-of-exemplars (NME). NME first computes a "class mean vector" for each class encountered so far. This essentially represents the average feature representation of all the exemplars belonging to that class. The calculation involves summing the feature vectors ($\Phi(x)$) of all training instances in a class and dividing by the total number of instances.

$$\mu_y = \frac{1}{|\epsilon_y|} \sum_{x \in \epsilon_y} \Phi(x) \quad (3.5)$$

where y is a class in the observed classes $1, 2, \dots, t$. Then, for a new unseen instance, NME computes its feature vector and assigns the class label whose mean vector is

closest to the new instance's feature vector:

$$y = \operatorname{argmin}_{y=1,2,\dots,t} \|\Phi(x) - \mu_y\| \quad (3.6)$$

This approach essentially classifies new data points based on their similarity to the representative exemplars of each class which can offer advantages in the context of CIL. The traditional approach relies on the model's final output score, which is sensitive to changes in the feature extractor Φ . When the model learns new data and updates the FC layer $\mathcal{F}$, the feature extractor Φ also needs to adapt. This can disrupt the way the model interprets features learned from previous data. In contrast, NME utilizes class mean vectors that adapt alongside changes in the feature extractor. While this adaptation in NME does lead to some shift in classification boundaries, it offers a key advantage: the model maintains a connection to previously learned information, mitigating the issue of catastrophic forgetting in CIL.

3.2.3 DER: Dynamic Expandable Representation

In the paper "DER: Dynamically Expandable Representation for Class Incremental Learning" by Yan et al. (2021) [13], the authors address the challenge of catastrophic forgetting in neural networks during incremental learning, where new classes are introduced over time. Their proposed solution leverages network expansion. When new classes are encountered, the model adds new feature extractors to learn representations specific to those classes. Crucially, the previously learned network remains frozen, which helps preserve knowledge of the previously encountered classes. This approach aims to strike a balance between adapting to new information and retaining past knowledge.

DER expands a new backbone when training a new tasks and update the FC layer $\mathcal{F}$ at the end of each task. At step t , the model's feature extractor Φ_t is build by expanding the feature extractor Φ_{t-1} with a newly created backbone $\mathcal{B}$. Specifically, given an connection x , the feature extracted by Φ_t is obtained by concatenation as follows

$$\Phi_t(x) = [\overline{\Phi_{t-1}}(x), \mathcal{B}(x)] \quad (3.7)$$

To reduce model's forgetting, the parameter of the last feature extractor $\overline{\Phi_{t-1}}$ is freeze. Since this component remains unchanged during subsequent learning stages, the features learned for previous tasks are preserved, preventing them from being overwritten by new information. The fully connected layer $\mathcal{F}$ is created with new shape every task. Further more, weights of $\mathcal{F}_t$ is initialized from $\mathcal{F}_{t-1}$ for fast adaptation and forward transfer. Now the loss function of the model at training task t

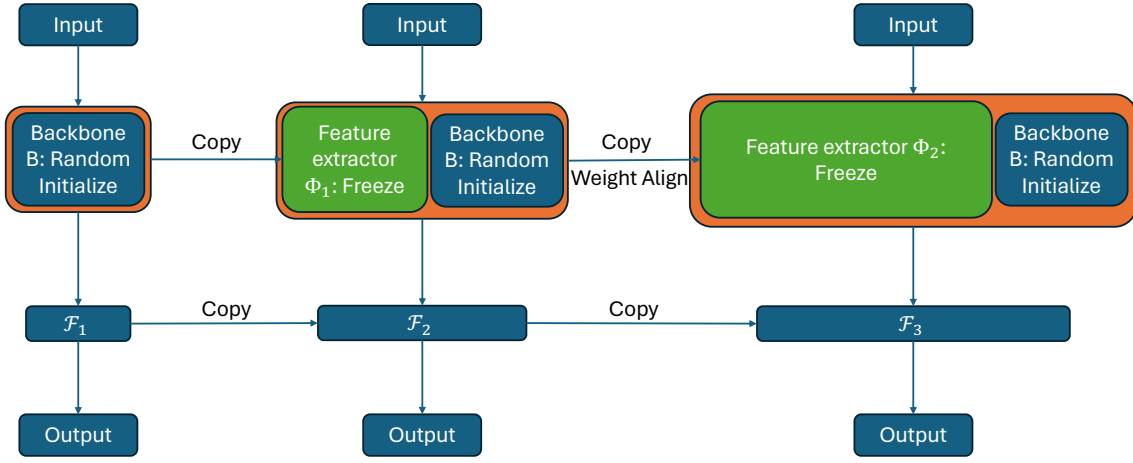


Figure 3.3: DER Flow

is:

$$L_{main} = -y * \log(\mathcal{S}(\mathcal{F}_t(\Phi_t(x)))) \quad (3.8)$$

The authors also introduce an auxiliary loss L_{aux} to enforce the network to learn the diverse and discriminative features for new concept. To encourage the network to learn features to discriminate between old and new concepts, they adding a aux target where its label space is $|Y_t| + 1$ including the new category set Y_t and the other class by treating all old classes as one category. The final loss function L_{total} is:

$$L_{total} = L_{main} + L_{aux} \quad (3.9)$$

3.2.4 MEMO: Memory Efficient Expandable Model

The model from paper A Model or 603 Exemplar: toward memory efficient class-incremental learning expands word of the paper in limited memory scenario. It argue that instead of expanding models every time facing new task, it only expands certain part of the model. Based on their observations that in a network, shallow layers capture general feature, while deeper layers handle more specific details for each classes, they introduces MEMO (Memory Efficient Expandable Model). MEMO builds upon a share based of general features learned from shallow layers. It then expands with specialized layers for new classes. Concretely, the model structure was redefined by decomposing the embedding layer module into specialized and generalized blocks. Specialized blocks correspond to the deep layers in the network, while generalized blocks correspond to the rest of the shallow layers. MEMO proposes to decouple the backbone at the middle level:

$$\Phi(x) = \Phi_s(\Phi_g(x)) \quad (3.10)$$

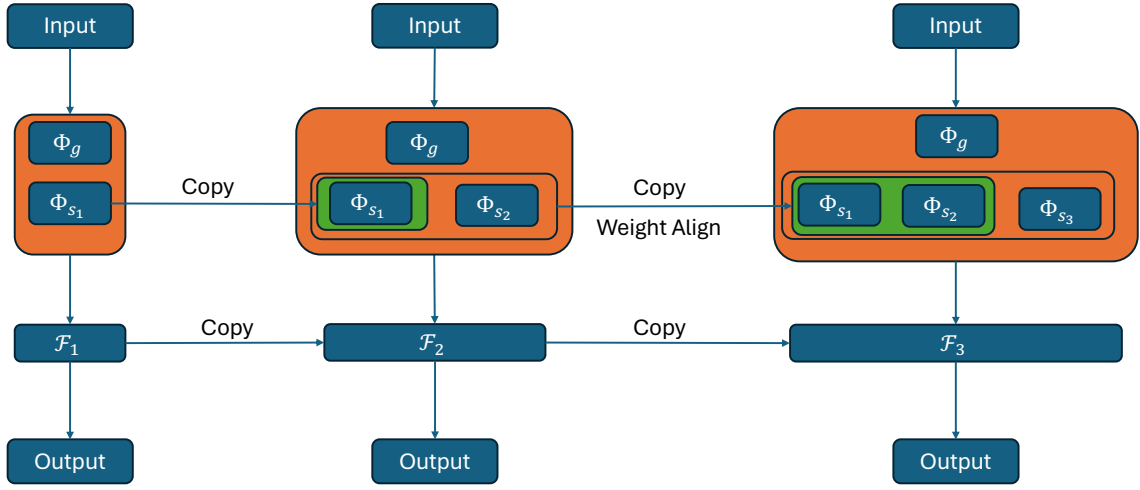


Figure 3.4: MEMO Flow

where specialized block Φ_s corresponds to the deep layer in the network, while generalized block Φ_g corresponds to the rest shallow layers. Compared to DER, MEMO only expands specialized blocks Φ_s .

Their research also examines the effects of freezing versus unfreezing the generalized layer. The findings indicate that freezing the generalized layer is more effective when training on a large number of base classes, while unfreezing it is beneficial when training from scratch.

3.2.5 Benchmark dataset

The models are experimented on four datasets: KDD99, CIC-IDS-2017, ToN-IoT-Network and UNSW-NB15 dataset. All of them are publicly available and widely used in Network Intrusion Detection Domain.

a, KDD99 Dataset

The KDD99 dataset, a widely recognized benchmark for evaluating intrusion detection systems [20], emerged from the 1990s KDD Cup, a competition focused on building network intrusion models. This competition leveraged data collected during the 1998 DARPA program. Here, MIT Lincoln Labs simulated a real Air Force Local Area Network (LAN) over nine weeks, deliberately introducing various attacks while capturing the raw network traffic. This raw data was then transformed into a collection of around five million individual connection records. Each record represents a data exchange between two IP addresses using a specific protocol. Each record is labeled as either "normal" or a specific type of attack, making it a valuable resource for training and assessing the performance of intrusion detection systems. KDD99 dataset categorizes network connection features into three distinct groups:

Table 3.1: Features set of KDD99 dataset

duration	su-attempted	same-srv-rate
protocol-type	num-root	diff-srv-rate
service	num-file-creations	srv-diff-host-rate
flag	num-shells	dst-host-count
src-bytes	num-access-files	dst-host-srv-count
dst-bytes	num-outbound-cmds	dst-host-same-srv-rate
land	is-host-login	dst-host-diff-srv-rate
wrong-fragment	is-guest-login	dst-host-same-src-port-rate
urgent	count	dst-host-srv-diff-host-rate
hot	srv-count	dst-host-serror-rate
num-failed-logins	serror-rate	dst-host-srv-serror-rate
logged-in	srv-serror-rate	dst-host-rerror-rate
num-compromised	rerror-rate	dst-host-srv-rerror-rate
root-shell	srv-rerror-rate	outcome

- **Basic features:** Encompasses all directly extractable attributes from a TCP/IP connection.
- **Traffic features:** These features, computed within a designated window interval, further analyze network activity for anomaly detection. They are further subdivided into same host features and same service features.
- **Content features:** Those features were extracted by using domain knowledge for detecting suspicious behaviors in data portions of the packages.

The feature set of the KD99 dataset is shown in Table 3.1. The four main categories of attacks and the 24 specific attack types are listed in Table 3.2 and the number of instances is detailed in Table 3.3.

Table 3.2: KDD99's category and related attack type

Category	Attack
DOS	Back, Pod, Land, Teardrop, Smurf, Neptune
R2L	ftp_write, guess_passwd, imap, multihop, phf, spy, warezclient, warezmaster
U2R	perl, buffer_overflow, loadmodule, rootkit,
Probing	Ipsweep, nmap, portsweep

Table 3.3: Number of instances per class of KDD99 dataset

Class	Number of instaces
smurf.	280790
neptune.	107201
normal.	97278
back.	2203
satan.	1589
ipsweep.	1247
portsweep.	1040
warezclient.	1020
teardrop.	979
pod.	264
nmap.	231
guess_passwd.	53
buffer_overflow.	30
land.	21
warezmaster.	20
imap.	12
rootkit.	10
loadmodule.	9
ftp_write.	8
multihop.	7
phf.	4
perl.	3
spy.	2

b, CIC-IDS-2017 dataset

The Canadian Institute for Cybersecurity (CIC) recognizes the significant limitations of many networks intrusion detection (IDS) dataset since 1998. These limitations often include outdated attacks models, limited traffic diversity, incomplete attack coverage and lack of feature sets and metadata. To address these shortcomings, CIC has developed the CIC-IDS-2017 dataset. This comprehensive dataset offers several key advantages: Real-World Reflectance, Network Insights, Naturalistic Background Traffic. The network traffic was captured over a five-day period, starting at 9 a.m. on Monday, July 3, 2017, and concluding at 5 p.m. on Friday, July 7, 2017. Monday's capture exclusively comprised benign traffic, while a range of attacks was executed during working hours on Tuesday, Wednesday, Thursday, and Friday. These attacks included Brute Force FTP, Brute Force SSH, Dos, Heartbleed, Web Attack, Infiltration, Botnet and DDoS. The CIC-IDS-2017 dataset provides a rich and detailed characteristics of network traffic through the analysis of network flows with CICFlowMeter. This analysis results in a comprehensive set

of 84 features that capture various aspects of network activity. These features can be grouped into several key categories. First, fundamental flow information captures essential details like IP addresses, timestamps, and protocols used. Second, packet-level insights delve deeper by examining the number of packets exchanged and their size, revealing the nature of communication (e.g., bulk data transfer vs frequent small messages). Third, flow-specific statistics provide both byte-level details on data volume and direction, as well as time-based information on flow duration and idle periods. Fourth, inter-arrival time patterns analyze the time gaps between packets. Finally, the distribution of flags (SYN, FIN, RST, PSH, ACK) within a flow offers insights into the communication protocol and helps identify deviations from standard patterns that might suggest suspicious behavior. The feature set of the CIC-IDS-2017 dataset is shown in Table 3.4 and the number of instances is detailed in Table 3.5.

Table 3.4: Features set of CIC-IDS-2017 dataset

Flow ID	Bwd IAT Total	Fwd Header Length.1 *
Source IP	Bwd IAT Mean	Fwd Avg Bytes/Bulk
Source Port	Bwd IAT Std	Fwd Avg Packets/Bulk
Destination IP	Bwd IAT Max	Fwd Avg Bulk Rate
Destination Port	Bwd IAT Min	Bwd Avg Bytes/Bulk
Protocol	Fwd PSH Flags	Bwd Avg Packets/Bulk
Timestamp	Bwd PSH Flags	Bwd Avg Bulk Rate
Flow Duration	Fwd URG Flags	Subflow Fwd Packets
Total Fwd Packets	Bwd URG Flags	Subflow Fwd Bytes
Total Backward Packets	Fwd Header Length	Subflow Bwd Packets
Total Length of Fwd Packets	Bwd Header Length	Subflow Bwd Bytes
Total Length of Bwd Packets	Fwd Packets/s	Init-Win-bytes-forward
Fwd Packet Length Max	Bwd Packets/s	Init-Win-bytes-backward
Fwd Packet Length Min	Min Packet Length	act-data-pkt-fwd
Fwd Packet Length Mean	Max Packet Length	min-seg-size-forward
Fwd Packet Length Std	Packet Length Mean	Active Mean
Bwd Packet Length Max	Packet Length Std	Active Std
Bwd Packet Length Min	Packet Length Variance	Active Max
Bwd Packet Length Mean	FIN Flag Count	Active Min
Bwd Packet Length Std	SYN Flag Count	Idle Mean
Flow Bytes/s	RST Flag Count	Idle Std
Flow Packets/s	PSH Flag Count	Idle Max
Flow IAT Mean	ACK Flag Count	Idle Min
Flow IAT Std	URG Flag Count	Label
Flow IAT Max	CWE Flag Count	
Flow IAT Min	ECE Flag Count	
Fwd IAT Total	Down/Up Ratio	
Fwd IAT Mean	Average Packet Size	
Fwd IAT Std	Avg Fwd Segment Size	
Fwd IAT Max	Avg Bwd Segment Size	
Fwd IAT Min		

c, TON-IoT dataset

The TON-IoT dataset presents a valuable resource for evaluating the efficiency of Artificial Intelligent (AI) powered cybersecurity solution in the context of Internet of Things. This rich repository comprises heterogeneous data captured from a meticulously crafted test bed encompassing diverse components like virtual machines, sensor-equipped physical systems, cloud platforms, etc. Its comprehensive coverage of real-world cyberattacks, ranging from denial-of-service attack to ransomware, makes it particularly relevant for developing and refining robust threat detection models. The directories of the TON-IoT datasets include IoT datasets,

Table 3.5: Number of instances per class of CIC-IDS-2017 dataset

Class	Number of instaces
BENIGN	2273097
DoS Hulk	231073
PortScan	158930
DDoS	128027
DoS GoldenEye	10293
FTP-Patator	7938
SSH-Patator	5897
DoS slowloris	5796
DoS Slowhttptest	5499
Bot	1966
Web Attack - Brute Force	1507
Web Attack - XSS	652
Infiltration	36
Web Attack - Sql Injection	21
Heartbleed	11

Network datasets, Linux datasets and Window datasets. In my experiment, I use Network Dataset, which were collected in the package capture (pcap) formats and processed by network analysis tool ZEEK. The feature set of the ToN-IoT-Network dataset is shown in Table 3.7 and the number of instances is detailed in Table 3.6.

Table 3.6: Number of instances per class of Ton-iot-network dataset

Class	Number of instaces
normal	50000
backdoor	20000
ddos	20000
dos	20000
injection	20000
password	20000
ransomware	20000
scanning	20000
xss	20000
mitm	1043

Table 3.7: Features set of Ton-Iot-Network dataset

ts	dst_ip_bytes	http_trans_depth
src_ip	dns_query	http_method
src_port	dns_qclass	http_uri
dst_ip	dns_qtype	http_version
dst_port	dns_rcode	http_request_body_len
proto	dns_AA	http_response_body_len
service	dns_RD	http_status_code
duration	dns_RA	http_user_agent
src_bytes	dns_rejected	http_orig_mime_types
dst_bytes	ssl_version	http_resp_mime_types
conn_state	ssl_cipher	weird_name
missed_bytes	ssl_resumed	weird_addl
src_pkts	ssl_established	weird_notice
src_ip_bytes	ssl_subject	label
dst_pkts	ssl_issuer	type

d, UNSW-NB15 dataset

UNSW-NB15 dataset is a widely used dataset for network intrusion detection system. Developed within the Cyber Range Lab of UNSW Canberra, it was made through the utilization of the IXIA PerfectStorm tool, yielding a corpus of raw traffic up to 100 GB in pcap file format, captured via the tcpdump tool. Subsequently, conversion of the raw traffic file into a structured dataset was achieved employing a combination of two network monitoring tools: ARGUS and BRO, and addition algorithms. This dataset includes nine distinct attack categories, including Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. The feature set of the UNSW-NB15 dataset is shown in Table 3.8 and the number of instances is detailed in Table 3.9.

Table 3.8: Features set of UNSW-NB15 dataset

id	dloss	response_body_len
dur	sinpkt	ct_srv_src
proto	dinpkt	ct_state_ttl
service	sjit	ct_dst_ltm
state	djit	ct_src_dport_ltm
spkts	swin	ct_dst_sport_ltm
dpkts	stepb	ct_dst_src_ltm
sbytes	dtcpb	is_ftp_login
dbytes	dwin	ct_ftp_cmd
rate	tcprtt	ct_flw_http_mthd
sttl	synack	ct_src_ltm
dttl	ackdat	ct_srv_dst
sload	smean	is_sm_ips_ports
dload	dmean	attack_cat
sloss	trans_depth	label

Table 3.9: Number of instances per class of UNSW-NB15 dataset

Class	Number of instaces
Normal	56000
Generic	40000
Exploits	33393
Fuzzers	18184
DoS	12264
Reconnaissance	10491
Analysis	2000
Backdoor	1746
Shellcode	1133
Worms	130

e, Synthetic dataset

The dataset generated using the `make_classification` function in 3.1 is a synthetic dataset designed to evaluate the performance of classification algorithms. This dataset comprises 10,000 samples, each characterized by 120 features. Among these features, 15 are informative, contributing directly to the classification task, while the remaining features do not contribute to class separation. The dataset is structured to have 10 distinct classes, each represented by a single cluster of data points. The `class_sep` parameter is set to 6, ensuring that the classes are well-separated and thus facilitating high classification accuracy. Additionally, the dataset is free from noise, as indicated by the `flip_y=0.0` parameter, which ensures that all labels are correctly assigned. This clean and well-structured dataset, with

no redundant features, provides an ideal testbed for evaluating the capabilities of various CIL methods and techniques in a controlled setting.

Listing 3.1: Code for generating Systhesis Dataset

```
X, y = make_classification(  
    random_state=42, n_classes=10, n_samples=10000,  
    n_informative=15, n_features=120, n_clusters_per_class=1,  
    class_sep=6, n_redundant=0, flip_y=0.0)
```

f, t-SNE visualization

I employed the t-SNE visualization technique [21] to illustrate my dataset. Dimension reduction was performed using the Python scikit-learn module [22], and the visualizations were generated with the Plotly library [23]. For each dataset class, up to 1000 instances were used. The resulting visualizations are shown in Figures 3.5, 3.6, 3.7, 3.8, 3.9. These visualizations indicate that the dataset is not linearly separable but can still be predicted using ML/DL methods. Some classes from the KDD99 and CIC-IDS-2017 datasets are straightforward to classify. However, the UNSW-NB15 and ToN-IoT-Network datasets present a much greater challenge for prediction.

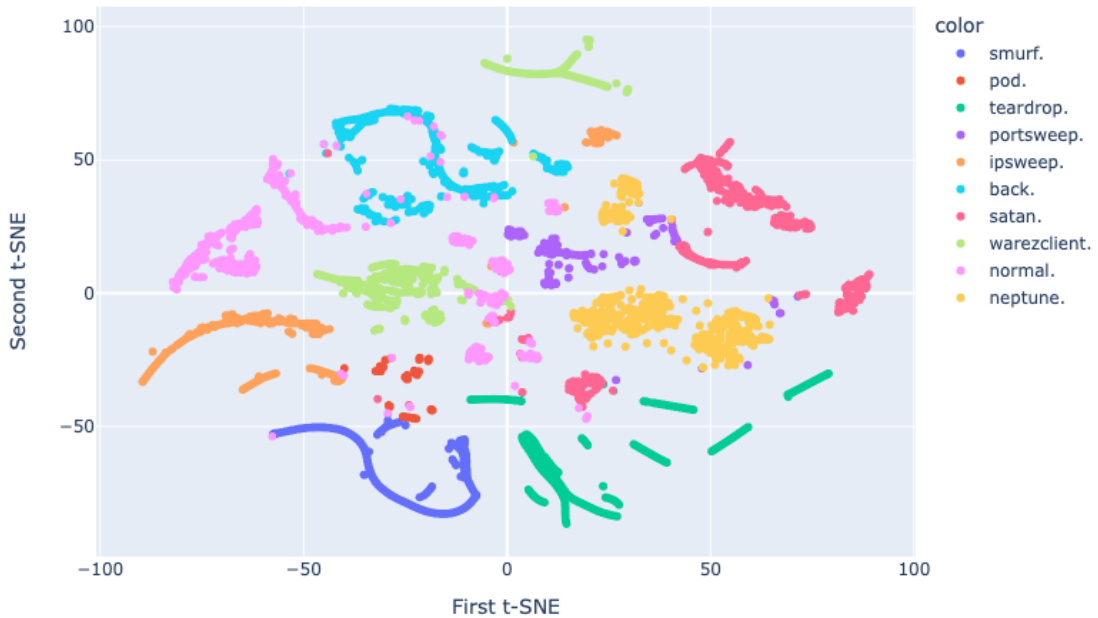


Figure 3.5: t-SNE visualization of KDD99 dataset

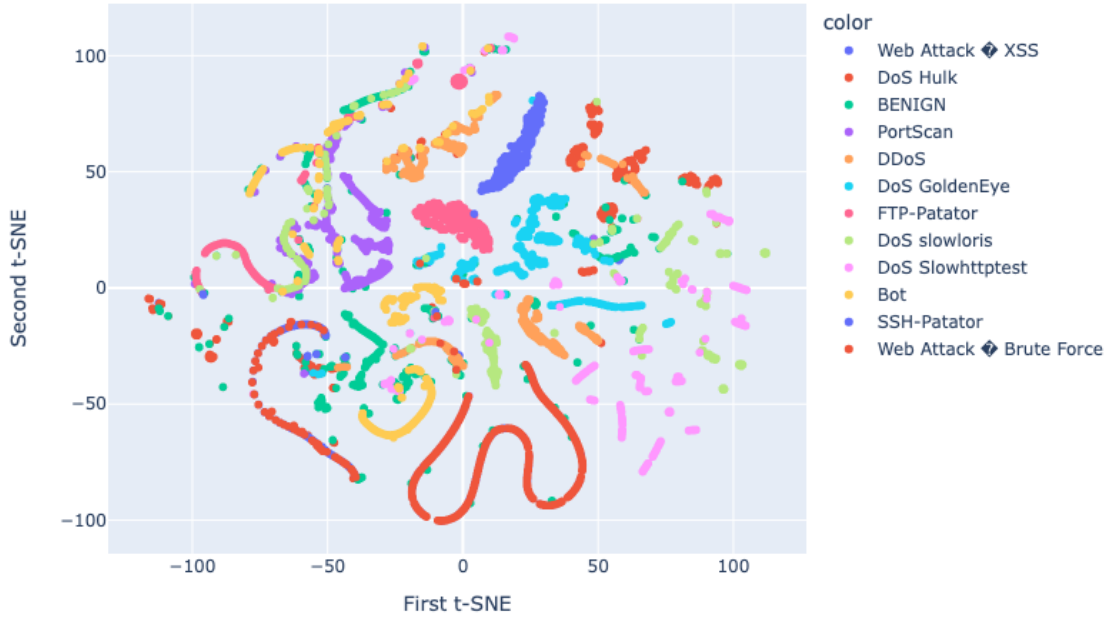


Figure 3.6: t-SNE visualization of CIC-IDS-2017 dataset

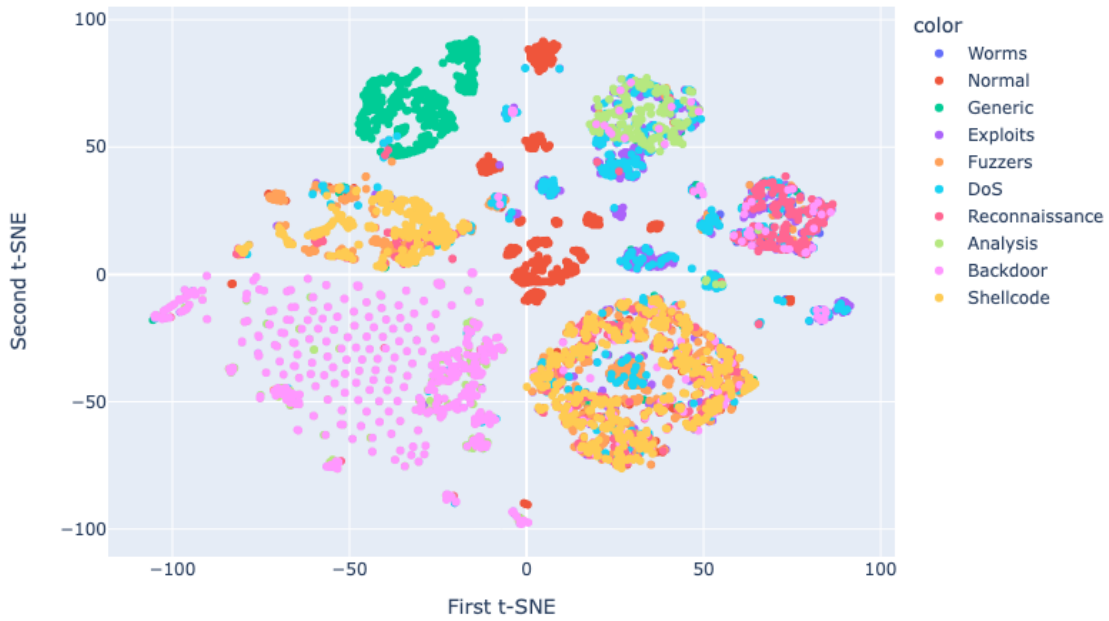


Figure 3.7: t-SNE visualization of UNSW-NB15 dataset

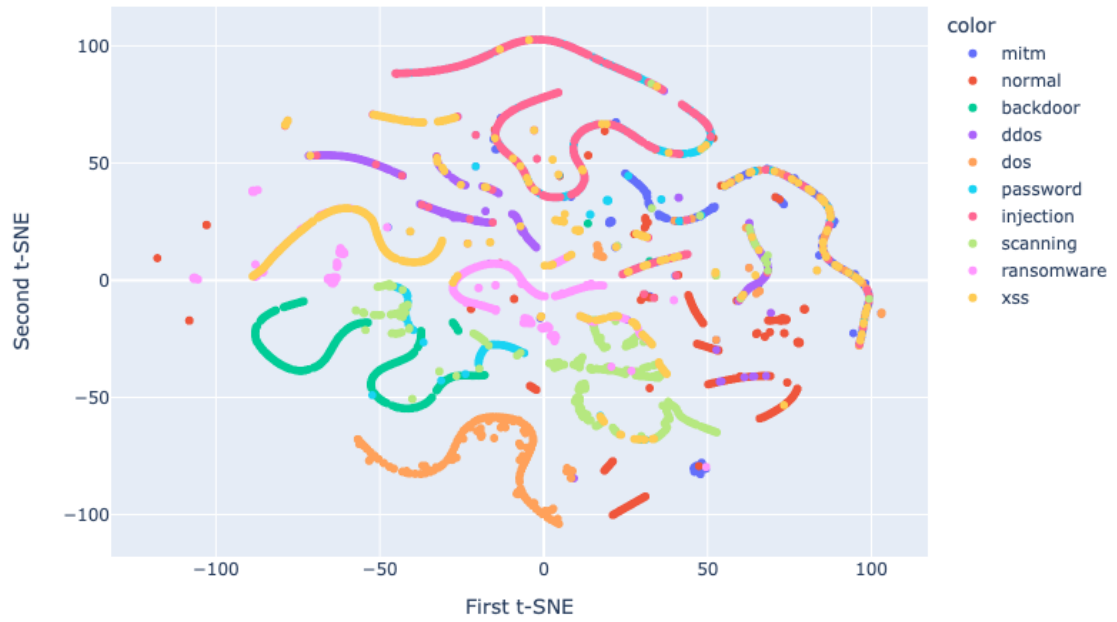


Figure 3.8: t-SNE visualization of ToN-IoT-Network dataset

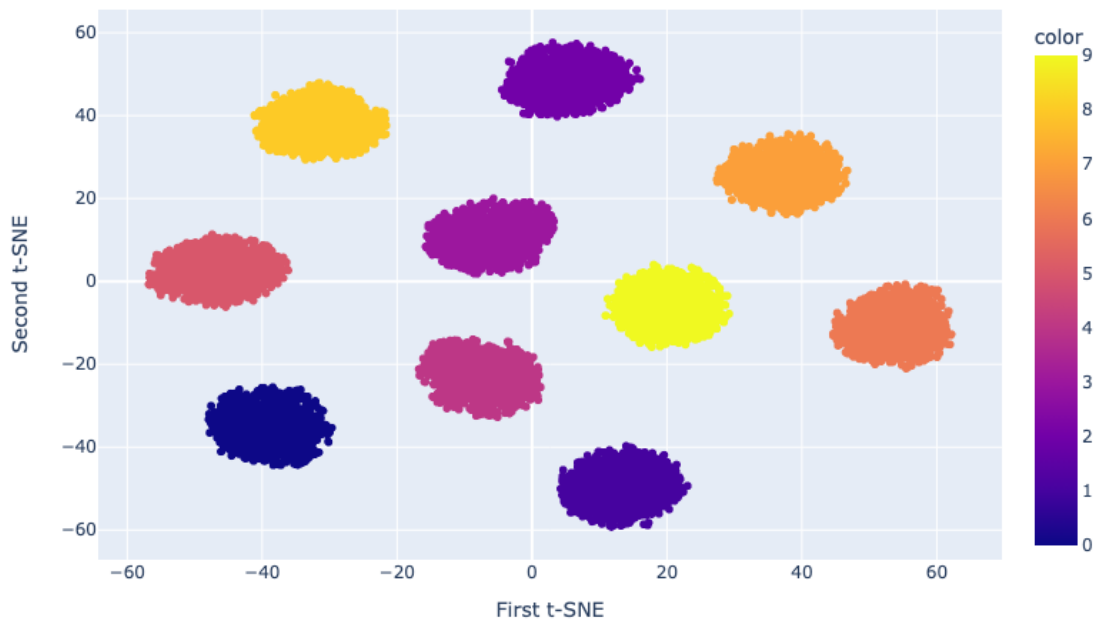


Figure 3.9: t-SNE visualization of Synthetic dataset

CHAPTER 4. NUMERICAL RESULTS

In this chapter, I conduct a thorough analysis of the methods implemented for incremental learning in Network Intrusion Detection Systems (NIDS). I systematically evaluate various methodologies using multiple datasets, such as KDD99, CIC-IDS-2017, ToN-IoT-Network, and UNSW-NB15. I begin by explaining the metrics used in the experiments. Following this, I explain the implementation details, including dataset processing, model architecture, and training parameters. I then examine each experiment in detail, assessing the performance of all methods, the effects of exemplar size, the influence of the temperature hyperparameter in knowledge distillation-based methods, and the impact of the NME classifier.

4.1 Evaluation Parameters

This thesis employs two metrics used by Zhou et al. (2023) [24] to assess the performance of different methods: Last Accuracy and Average Accuracy. Last Accuracy evaluates the model's accuracy on the most recent task it encountered during training. Average Accuracy is the mean accuracy of the model across all tasks it has been trained on, up to the current point. This metric provides a view of the model's overall performance, reflecting its ability to generalize and maintain knowledge over time.

4.2 Data Preprocessing

4.2.1 Feature Encoding

One-Hot Encoding was used for categorical features. One-Hot Encoding creates a separate binary dimension for each category within a feature. Each dimension takes a value of 1 for its corresponding category and 0 for all others.

After encoding the categorical features, Min-Max normalization was applied to scale the features. Min-Max normalization scales the feature values to a range of [0, 1]. The Min-Max normalized value x' of a feature is calculated using the formula:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (4.1)$$

where x_{min} and x_{max} are the minimum and maximum values of the feature, respectively.

By the end, the dataset was divided into train and test data with a ratio of 70:30, where 30% was kept aside for testing model's generalizability. All of data preprocessing steps are performed on pandas dataframe [25].

4.2.2 Feature Processing

- **Duplication removal:** I remove all duplicate records from all dataset.
- **NaN value removal:** I remove all Null records from all dataset (record with missing features) as these make up only a small percentage of data.

4.2.3 Dataset Processing

a, CIC-IDS-2017 dataset

All of dataset file is used. I use all file from file MachineLearningCSV.zip. The file include attack from all days. The name of file are:

- Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv
- Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv
- Friday-WorkingHours-Morning.pcap_ISCX.csv
- Monday-WorkingHours.pcap_ISCX.csv
- Thursday-WorkingHours-Afternoon-Infiltration.pcap_ISCX.csv
- Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv
- Tuesday-WorkingHours.pcap_ISCX.csv
- Wednesday-workingHours.pcap_ISCX.csv

I concatenated all these files and removed the "Fwd Header Length.1" column as it duplicates the "Fwd Header Length" column. Additionally, features with only one unique value were removed: ["Bwd PSH Flags", "Bwd URG Flags", "Fwd Avg Bytes/Bulk", "Fwd Avg Packets/Bulk", "Fwd Avg Bulk Rate", "Bwd Avg Bytes/Bulk", "Bwd Avg Packets/Bulk", "Bwd Avg Bulk Rate"]. Classes with fewer than 100 instances, such as Infiltration, SQL Injection, and Heartbleed, were also excluded.

b, Ton-Iot-Network dataset

I use file train_test_network.csv from folder Train_Test_Network_Dataset. Features [src_ip, dst_ip, dns_query] are dropped.

c, KDD99 dataset

In this experiment, I use file kddcup.data_10_percent.gz for training and testing purpose. This is a 10% subset of the full dataset. This dataset contains 494021 records in totals, in which 280790 records recognized as smurf attack. Classes with fewer than 200 instances are excluded.

d, UNSW-NB15 dataset

I use file UNSW_NB15_training_set.csv for training and testing purpose. Features ["id", "label"] are dropped.

4.3 Implementation Detail

4.3.1 Network Architecture

I use the same backbone architectures across all datasets. The backbone comprises three layers, based on architecture size detailed in paper "Deep Learning Approach for Intrusion Detection System" [2] and by considering the continuous expansion for DER and MEMO models. LWF and ICARL models have a constant configuration of two backbones: one for previous tasks and another for the current task. For DER, the network adds a new backbone of the same size whenever encounter new task. In MEMO, the first layer is the generalized layer, while the second layer is the specialized. When facing a new task, a new backbone of the same size is added to the network for DER. In the case of MEMO, the first layer is considered the generalized layer, and the second layer is the specialized layer and only the second layer is expanding. In all my experiments with the MEMO model, I chose not to freeze the generalized layer. For all models, the fully connected (FC) layer expands upon encountering new tasks.

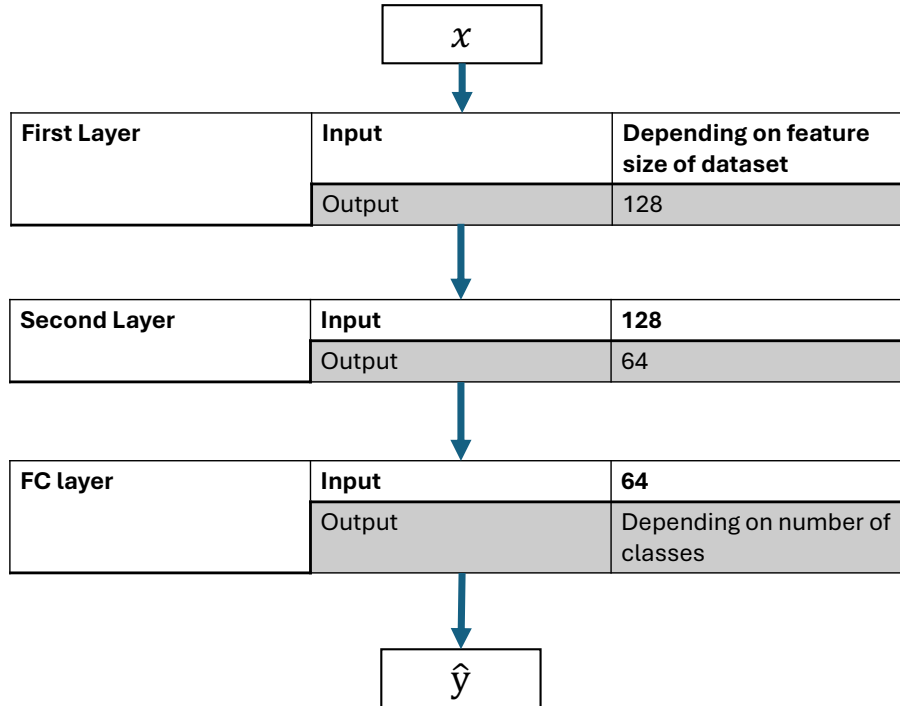


Figure 4.1: Model Architecture

4.3.2 Training Parameter setting

- **Epochs:** 100. Epochs is the number of training epochs per task.

- **Optimizer:** I use Stochastic Gradient Descent as the optimizer.
- **Learning rate:** 0.1. Learning rate is a parameter of SGD. Learning rate determines how quickly a model adjusts its internal parameter during training.
- **Scheduler:** MultiStepLR is used as learning scheduler. MultiStepLR decays the learning rate of each parameter group by gamma once the number of epochs reaches one of the milestones. This makes sure the learning rate is not too big or too small. In my experiment I use milestone [25, 50, 75] and gamma = 0.1
- **Training order:** I train with classes from having most instances to having least instances.
- **Batch size:** 32. This is a good batch size to start with. Small batch size allow models to update more frequently and can provide better accuracy.
- **Increment Classes per Task:** 2. For each new task, two additional classes will be trained.
- **Base Classes Trained:** 2. This is the number of classes that were trained during the initial task.

4.4 Comparison of Implemented Methods Among Different Dataset

In this experiment, I evaluated the performance of different methods across various datasets, using Last Accuracy and Average Accuracy as the evaluation metrics. The baseline model is Finetune, which has a single backbone and suffers from catastrophic forgetting. For the DER method, I tested two models: one using only the cross-entropy loss function (as described in equation 3.8), and another that combines the cross-entropy loss with an auxiliary loss (as in equation 3.9). Similarly, for ICARL, I evaluated two models: one using the standard cross-entropy loss function and another incorporating knowledge distillation loss (as in equation 3.2).

Table 4.1: Evaluation on KDD99 dataset

Methods	Last Accuracy	Average Accuracy
Finetune - Normal Loss	0.43	20.75
DER - Normal Loss	88.94	83.28
DER - Aux Loss	98.97	98.95
MEMO	91.51	95.88
LWF	58.62	68.3
ICARL KD Loss	92.91	96.59
ICARL Normal Loss	90.27	96.18

Table 4.2: Evaluation on CIC-IDS-2017 dataset

Methods	Last Accuracy	Average Accuracy
Finetune - Normal Loss	0.23	22.03
DER - Normal Loss	23.41	29.76
DER - Aux Loss	38.32	32.27
MEMO	22.98	25.96
LWF	0.23	13.15
ICARL KD Loss	22.72	49.93
ICARL Normal Loss	39.64	50.44

Table 4.3: Evaluation on ToN-IoT-Network dataset

Methods	Last Accuracy	Average Accuracy
Finetune - Normal Loss	7.85	38.26
DER - Normal Loss	17.89	40.27
Der - Aux Loss	17.8	40.25
MEMO	8.15	38.32
LWF	22.75	52.06
ICARL KD Loss	0.58	66.24
ICARL Normal Loss	10.85	68.58

Table 4.4: Evaluation on UNSW-NB15 dataset

Methods	Last Accuracy	Average Accuracy
Finetune - Normal Loss	1.07	23.02
DER - Normal Loss	4.22	24.33
DER - Aux Loss	15.89	27.92
MEMO	1.64	23.13
LWF	1.07	22.46
ICARL KD Loss	4.97	30.61
ICARL Normal Loss	0.37	32.09

Table 4.5: Evaluation on Synthetic dataset

Methods	Last Accuracy	Average Accuracy
Finetune - Normal Loss	20.2	45.9
DER - Normal Loss	99.45	99.84
DER - Aux Loss	100	100
MEMO	99.6	99.88
LWF	25.75	50.21
ICARL KD Loss	99.95	99.99
ICARL Normal Loss	99.99	100

Tables 4.1, 4.2, 4.3, 4.4, 4.5 provide a comprehensive comparison of different methods evaluated across several datasets: KDD99, CIC-IDS-2017, ToN-IoT-Network, UNSW-NB15, and a synthetic dataset. Overall, all methods significantly

outperform the Finetune method, which is affected by catastrophic forgetting. Finetune models exhibit low final accuracy, as low as 0.43% on the KDD99 dataset and 0.23% on the CIC-IDS-2017 dataset, due to their tendency to predict classes from the most recently trained task, influenced by the order of training. DER and ICARL has strong performances among all the dataset and all evaluation. ICARL has strongest performance in all dataset, with highest Average Accuracy in CIC-IDS-2017, ToN-IoT-Network, and UNSW-NB15 dataset and second highest in KDD99 dataset after DER. For DER, using auxiliary loss enhances its performance. However, for ICARL, utilizing Knowledge Distillation Loss reduces the average accuracy. MEMO performs worse than DER in all cases. This can be attributed to the small model size and the differing characteristics of each attack type. Unfreezing the generalized layer causes feature drift, leading to catastrophic forgetting. For LWF, the performance is significantly worse compared to methods using exemplars. This highlights the importance of an exemplar set in Class Incremental Learning. However, when compared to Finetune, LWF achieves much better accuracy on the KDD99 dataset, ToN-IoT-Network dataset, and Synthetic Dataset. There is no improvement in accuracy on the UNSW-NB15 dataset, and the average accuracy is lower on the CIC-IDS-2017 dataset. An examination of the training process reveals that this is because using loss results in the models failing to learn the features of the new classes. Further details are discussed in Appendix A.

4.5 Comparison of Implemented Methods With Different Exemplar Size

This experiment investigates the effect of different exemplar sizes on model performance. The goal was to assess how varying the number of stored exemplars influences the accuracy and learning capability of various incremental learning methods. In this study, exemplar sizes of 1000, 500, 250, and 100 were used for three methods: DER, MEMO, and ICARL.

Table 4.6: Evaluation on KDD99 dataset, with different exemplar size

Methods	Last Accuracy	Average Accuracy
DER 1000 - Normal Loss	30.3	59.27
DER 1000 - Aux Loss	91.63	94.57
MEMO 1000	89.08	94.5
ICARL 1000 - KD Loss	89.5	96.39
ICARL 1000 - Normal Loss	89.96	95.58
DER 500 - Normal Loss	48.64	53.03
DER 500 - Aux Loss	88.5	93.02
MEMO 500	30	58.74
ICARL 500 - KD Loss	53.43	80.43
ICARL 500 - Normal Loss	90.13	84.27
DER 250 - Normal Loss	0.42	28.01
DER 250 - Aux Loss	1.08	51.59
MEMO 250	1.21	21.02
ICARL 250 - KD Loss	60.06	62.86
ICARL 250 - Normal Loss	69.96	65.06
DER 100 - Normal Loss	0.42	27.23
DER 100 - Aux Loss	1.6	33.73
MEMO 100	0.43	20.75
ICARL 100 - KD Loss	27.21	67.11
ICARL 100 - Normal Loss	36.67	62.79

Table 4.8: Evaluation on Ton-IoT-Network dataset, with different exemplar size

Methods	Last Accuracy	Average Accuracy
DER 1000 - Normal Loss	7.85	38.26
DER 1000 - Aux Loss	7.85	38.26
MEMO 1000	8.15	38.32
ICARL 1000 - KD Loss	15.85	70.57
ICARL 1000 - Normal Loss	8.03	66.01
DER 500 - Normal Loss	7.92	38.27
DER 500 - Aux Loss	8.4	38.37
MEMO 500	8.15	38.32
ICARL 500 - KD Loss	16.23	70.94
ICARL 500 - Normal Loss	13.24	71.19
DER 250 - Normal Loss	8.05	38.3
DER 250 - Aux Loss	8.4	38.37
MEMO 250	8.15	38.32
ICARL 250 - KD Loss	19.08	71.62
ICARL 250 - Normal Loss	19.28	73.2
DER 100 - Normal Loss	8.1	38.31
DER 100 - Aux Loss	8.43	38.38
MEMO 100	5.24	23.86
ICARL 100 - KD Loss	21.98	72.49
ICARL 100 - Normal Loss	21.09	73.06

Table 4.7: Evaluation on CIC-IDS-2017 dataset, with different exemplar size

Methods	Last Accuracy	Average Accuracy
DER 1000 - Normal Loss	23.05	26.2
DER 1000 - Aux Loss	27.84	27.38
MEMO 1000	22.94	25.95
ICARL 1000 - KD Loss	41.46	54.73
ICARL 1000 - Normal Loss	35.91	51.79
DER 500 - Normal Loss	18.91	25.23
DER 500 - Aux Loss	22.95	25.98
MEMO 500	0.85	22.27
ICARL 500 - KD Loss	28.11	50.85
ICARL 500 - Normal Loss	25.14	47.74
DER 250 - Normal Loss	0.23	22.16
DER 250 - Aux Loss	6.91	23.31
MEMO 250	0.23	22.16
ICARL 250 - KD Loss	12.04	48.18
ICARL 250 - Normal Loss	26.68	51.75
DER 100 - Normal Loss	0.57	26.55
DER 100 - Aux Loss	0.23	23.2
MEMO 100	0.23	23.2
ICARL 100 - KD Loss	15.94	44.98
ICARL 100 - Normal Loss	17.24	42.97

Table 4.10: Evaluation on Synthetic dataset, with different exemplar size

Methods	Last Accuracy	Average Accuracy
DER 1000 - Normal Loss	99.95	99.99
DER 1000 - Aux Loss	100	99.96
MEMO 1000	99.55	99.84
ICARL 1000 - KD Loss	99.9	99.99
ICARL 100 - Normal Loss	99.85	99.97
DER 500 - Normal Loss	99.35	99.87
DER 500 - Aux Loss	100	100
MEMO 500	99.8	99.88
ICARL 500 - KD Loss	99.95	99.99
ICARL 500 - Normal Loss	99.7	99.93
DER 250 - Normal Loss	97.7	99.26
DER 250 - Aux Loss	100	99.82
MEMO 250	47.6	79.84
ICARL 250 - KD Loss	80.85	96.13
ICARL 250 - Normal Loss	41.5	75.35
DER 100 - Normal Loss	70.55	90.71
DER 100 - Aux Loss	87.2	96.3
MEMO 100	30.15	64.1
ICARL 100 - KD Loss	28.7	69.25
ICARL 100 - Normal Loss	30.3	61.2

Table 4.9: Evaluation on UNSW-NB15 dataset, with different exemplar size

Methods	Last Accuracy	Average Accuracy
DER 1000 - Normal Loss	1.07	23.2
DER 1000 - Aux Loss	4.22	24.32
MEMO 1000	5.24	23.86
ICARL 1000 - KD Loss	1.65	30.33
ICARL 1000 - Normal Loss	2.79	36.24
DER 500 - Normal Loss	1.07	23.1
DER 500 - Aux Loss	1.07	23.2
MEMO 500	1.07	23.02
ICARL 500 - KD Loss	5.09	32.85
ICARL 500 - Normal Loss	4.66	38.88
DER 250 - Normal Loss	1.07	23.07
DER 250 - Aux Loss	1.72	23.2
MEMO 250	1.09	23.03
ICARL 250 - KD Loss	18.56	41.75
ICARL 250 - Normal Loss	3.69	37.62
DER 100 - Normal Loss	1.07	23.07
DER 100 - Aux Loss	1.16	23.08
MEMO 100	1.16	23.06
ICARL 100 - KD Loss	9.09	32.68
ICARL 100 - Normal Loss	24.26	32.94

Table 4.6, 4.7, 4.8, 4.9, 4.10 evaluate the performance of DER, MEMO, and ICARL across all benchmark datasets. Generally, ICARL achieves the highest accuracy on most datasets with varying exemplar sizes. However, DER outperforms ICARL on the KDD99 dataset when the exemplar size is 1000 or larger and on the Synthetic dataset. This indicates that DER is superior to ICARL when the exemplar proportion is sufficient. Increasing the exemplar size usually boosts overall accuracy, as seen with the KDD99, CIC-IDS-2017, and Synthetic datasets. Additionally, DER with auxiliary loss outperforms DER with cross-entropy loss, though the same cannot be concluded for ICARL with KD Loss. For the ToN-IoT-Network and UNSW-NB15 datasets, increasing exemplar size does not improve accuracy, and performance varies significantly. This likely occurs because these datasets are challenging to classify correctly, and some classes are impossible to classify based on available features. MEMO consistently performs the worst for the same reasons as mentioned in the previous experiment.

4.6 Experiment with Temperature parameter in Knowledge Distillation-based methods

In this experiment, I change the temperature hyper parameter to see if the change in the evaluation results. I test the results on KDD99 dataset with exemplar size

500 and Synthetic dataset with exemplar size 100. The goal of this experiment was to investigate how the temperature hyper-parameter affects model performance. Specifically, it aimed to see if modifying the temperature parameter, which affects the knowledge distillation process, could improve the model capacity to remember information from previously learned classes while successfully incorporating data from new classes. The experiment was performed on the KDD99 and Synthetic dataset, using different temperature setting with two knowledge distillation-based methods: ICARL (which uses an exemplar set) and LWF (which does not use an exemplar set). The exemplar size was 500 for KDD99 dataset and 100 for Synthetic dataset.

Table 4.11: Evaluations of LWF and ICARL on Synthetic dataset with different temperature parameter

Methods	Temperature	Last Accuracy	Average Accuracy
LWF	0.25	9.55	32.4
LWF	0.5	9.55	41.23
LWF	1	20.2	45.9
LWF	2	20.2	45.9
LWF	4	20.2	45.9
LWF	8	20.2	45.9
ICARL	0.25	94.1	96.04
ICARL	0.5	96.95	98.53
ICARL	1	97.6	99.49
ICARL	2	80.85	96.13
ICARL	4	76.05	85.59
ICARL	8	51.45	81.38

Table 4.12: Evaluations of LWF and ICARL on KDD99 dataset with different temperature parameter

Methods	Temperature	Last Accuracy	Average Accuracy
LWF	0.25	0.32	20.38
LWF	0.5	0.32	20.52
LWF	1	0.43	20.75
LWF	2	0.43	20.75
LWF	4	0.43	20.75
LWF	8	0.43	20.75
ICARL	0.25	99.1	98.42
ICARL	0.5	92.77	97.03
ICARL	1	89.35	96.17
ICARL	2	89.5	96.39
ICARL	4	89.95	96.17
ICARL	8	89.97	95.93

Tables 4.11 and 4.12 present a comparative evaluation of LWF and ICARL across two distinct datasets, Synthetic and KDD99, under varying temperature parameters. For LWF, in both datasets, neither Last Accuracy nor Average Accuracy increases as the temperature T rises from 1 to 8, and accuracy decreases when T is lower than 1. This can be attributed to LWF’s lack of an exemplar set, making the knowledge distillation loss ineffective when learning models facing new classes. The decline in accuracy for $T < 1$ can be due to the dominance of the knowledge distillation loss over the normal loss, which hinders the model’s ability to learn new classes. In contrast, for ICARL, the highest accuracy on the Synthetic dataset occurs at a temperature of 1, while for the KDD99 dataset, the highest accuracy is achieved at a temperature of 0.25. In the case of the KDD99 dataset, the same reasoning applies: the dominance of the knowledge distillation loss leads the model to predict more instances of old classes. Additionally, KDD99 is a highly imbalanced dataset, with the training order progressing from the largest to the smallest classes. In conclusion, these experiments indicate that knowledge distillation has no impact when there are no instances from former classes, which is expected. When exemplars are present, as in the case of ICARL, the temperature parameter T affects accuracy; however, the effectiveness of the knowledge distillation loss remains uncertain. This ambiguity may arise from differences in implementation contexts. In the original paper, knowledge distillation is used to transfer knowledge from an ensemble of models or a larger model (such as one with dropout layers, which can be interpreted as an ensemble) to a smaller model. The same conclusion is reached by Eden Belouadah et al. [26], where the authors assess the role of knowledge distillation in their surveys. Many works have employed knowledge distillation in incremental learning, all drawing inspiration from ICARL. However, the comparisons by ICARL are not fair since some methods do not use memory while others utilize exemplars.

4.7 Evaluation of DER with NME Classifier

The previous experiments demonstrated that knowledge distillation loss, as implemented in LWF and ICARL, has minimal impact on model accuracy. However, when evaluating different exemplar sizes, ICARL consistently outperforms DER, despite both methods utilizing the same exemplar selection strategy: herding. This superior performance of ICARL may be attributed to its use of the mean of exemplars classifier. In this experiment, I investigate the performance of DER when employing Near Mean of Exemplars (NME) classifier. DER, incorporating an auxiliary loss, is tested across all datasets. For the KDD99, CIC-IDS-2017, ToN-IoT-Network, and UNSW-NB15 datasets, an exemplar size of 1000 is used, while

for the synthetic dataset, an exemplar size of 100 is utilized.

Table 4.13: Evaluations of DER on multiple datasets using NME Classifier

Classifier	Dataset	Last Accuracy	Average Accuracy
Normal	KDD99	91.63	94.57
NME	KDD99	97.15	97.88
Normal	CIC-IDS-2017	27.84	27.38
NME	CIC-IDS-2017	63.39	71.83
Normal	Ton-Iot-Network	7.85	38.26
NME	Ton-Iot-Network	94.1	95.19
Normal	UNSW-NB15	4.22	24.32
NME	UNSW-NB15	15.91	31.37
Normal	Synthetic	87.2	96.3
NME	Synthetic	98.85	99.77

The table presents the evaluation results of DER (with auxiliary loss) using the Nearest Mean of Exemplars (NME) classifier across multiple datasets: KDD99, CIC-IDS-2017, ToN-IoT-Network, UNSW-NB15, and synthetic dataset. For each dataset, the performance metrics, Last Accuracy and Average Accuracy, are compared between the normal classifier and the NME classifier. The results demonstrate that the NME classifier consistently outperforms the normal classifier across all datasets. The accuracy achieved by the DER with the Nearest Mean of Exemplars (NME) classifier surpasses that of the best model from previous experiments, ICARL, by a significant margin. This can be explained by the training process of DER with a normal classifier, where the final fully connected (FC) layers are updated. The predictions rely on these updated FC layers. In contrast, when using the NME classifier, only the backbone layers are utilized, and these layers are frozen when facing new tasks, preventing catastrophic forgetting. The use of NME fixes the issues related to the updating of the FC layers, resulting in superior performance that approaches the upper bound achieved by joint training models.

CHAPTER 5. CONCLUSIONS

5.1 Summary

This thesis has explored the application of Class Incremental Learning (CIL) techniques to Network Intrusion Detection System (NIDS). Traditional NIDS models are often failed to adapt to the continuously evolving threat landscape. By integrating CIL, I aim to create a system that incrementally learns and incorporates new attack patterns without requiring complete retraining on the entire dataset. My research has focused on evaluating various state-of-the-art models from the Image Classification domain such as Dynamic Expandable Representation (DER), Memory Efficient Model (MEMO), Learning Without Forgetting (LWF) and Incremental Classifier and Representation Learning (ICARL) in the context of NIDS. The experimental results, conducted across multiple benchmark datasets, including KDD99, CIC-IDS-2017, ToN-IoT-Network, UNSW-NB15 and a synthetic dataset, demonstrate the feasibility and potential benefits of applying the CIL techniques to NIDS. The results indicate that all methods outperform the standard model in incremental learning. However, their effectiveness heavily relies on the size of the exemplar set. This implies that each CIL-NIDS system should carefully determine the appropriate exemplar size for optimal performance and minimal memory usage. The best method, DER, combined with the NME classifier, can achieve very high accuracy. Nevertheless, DER's reliance on network expansion may limit its practicality in real-world applications. Knowledge Distillation appears to be ineffective in Class Incremental Learning scenarios. Additionally, the impact of noisy data on the learning process should be considered when implementing NIDS systems.

5.2 Suggestion for Future Works

- **Addressing Class Imbalance:** The experimental results indicate a strong correlation between exemplar size and model performance. When the exemplar size is small compared to the training set of new classes, it results in class imbalance issues. Future research could investigate methods to mitigate class imbalance, such as employing data augmentation techniques or resampling strategies.
- **Fair Evaluations of Dynamic Expanding Models:** Dynamic expanding models demonstrate superior performance compared to other models. However, the increase in weights associated with these models can raise concerns about fairness. Zhou et al. (2023) [24] conducted evaluations using the same memory settings on the CIFAR-10 and ImageNet datasets. These evaluations inher-

ently depend on factors such as the number of model parameters, input size, and exemplar size, which can lead to biased results. To ensure fair comparisons, dynamic models should be evaluated against a set of models trained separately on different tasks under CIL setting.

REFERENCE

- [1] E. Hodo, X. Bellekens, A. Hamilton, C. Tachtatzis, and R. Atkinson, “Shallow and deep networks intrusion detection system: A taxonomy and survey,” *arXiv preprint arXiv:1701.02145*, 2017.
- [2] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, “Deep learning approach for intelligent intrusion detection system,” *Ieee Access*, vol. 7, pp. 41 525–41 550, 2019.
- [3] M. Data and M. Aritsugi, “T-dfnn: An incremental learning algorithm for intrusion detection systems,” *IEEE Access*, vol. 9, pp. 154 156–154 171, 2021.
- [4] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, “Network intrusion detection system: A systematic study of machine learning and deep learning approaches,” *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, e4150, 2021.
- [5] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, “A deep learning approach to network intrusion detection,” *IEEE transactions on emerging topics in computational intelligence*, vol. 2, no. 1, pp. 41–50, 2018.
- [6] K. Ghanem, F. J. Aparicio-Navarro, K. G. Kyriakopoulos, S. Lambotharan, and J. A. Chambers, “Support vector machine for network intrusion and cyber-attack detection,” in *2017 sensor signal processing for defence conference (SSPD)*, IEEE, 2017, pp. 1–5.
- [7] M. Al-Qatf, Y. Lasheng, M. Al-Habib, and K. Al-Sabahi, “Deep learning approach combining sparse autoencoder with svm for network intrusion detection,” *Ieee Access*, vol. 6, pp. 52 843–52 856, 2018.
- [8] Y. Xiao, C. Xing, T. Zhang, and Z. Zhao, “An intrusion detection model based on feature reduction and convolutional neural networks,” *IEEE Access*, vol. 7, pp. 42 210–42 219, 2019.
- [9] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. Torr, “Riemannian walk for incremental learning: Understanding forgetting and intransigence,” in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 532–547.
- [10] J. Bang, H. Kim, Y. Yoo, J.-W. Ha, and J. Choi, “Rainbow memory: Continual learning with a memory of diverse samples,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 8218–8227.

- [11] H. Shin, J. K. Lee, J. Kim, and J. Kim, “Continual learning with deep generative replay,” *Advances in neural information processing systems*, vol. 30, 2017.
- [12] C. He, R. Wang, S. Shan, and X. Chen, “Exemplar-supported generative reproduction for class incremental learning,” in *BMVC*, vol. 1, 2018, p. 2.
- [13] S. Yan, J. Xie, and X. He, “Der: Dynamically expandable representation for class incremental learning,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 3014–3023.
- [14] F.-Y. Wang, D.-W. Zhou, H.-J. Ye, and D.-C. Zhan, “Foster: Feature boosting and compression for class-incremental learning,” in *European conference on computer vision*, Springer, 2022, pp. 398–414.
- [15] D.-W. Zhou, Q.-W. Wang, H.-J. Ye, and D.-C. Zhan, “A model or 603 exemplars: Towards memory-efficient class-incremental learning,” *arXiv preprint arXiv:2205.13218*, 2022.
- [16] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, *et al.*, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the national academy of sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [17] Z. Li and D. Hoiem, “Learning without forgetting,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 12, pp. 2935–2947, 2018. DOI: 10.1109/TPAMI.2017.2773081.
- [18] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [19] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, “Icarl: Incremental classifier and representation learning,” in *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, 2017, pp. 2001–2010.
- [20] A. Özgür and H. Erdem, “A review of kdd99 dataset usage in intrusion detection and machine learning between 2010 and 2015,” 2016.
- [21] L. Van der Maaten and G. Hinton, “Visualizing data using t-sne,” *Journal of machine learning research*, vol. 9, no. 11, 2008.
- [22] F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, “Scikit-learn: Machine learning in python,” *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.
- [23] C. Sievert, *Interactive web-based data visualization with R, plotly, and shiny*. Chapman and Hall/CRC, 2020.
- [24] D.-W. Zhou, Q.-W. Wang, Z.-H. Qi, H.-J. Ye, D.-C. Zhan, and Z. Liu, “Deep class-incremental learning: A survey,” *arXiv preprint arXiv:2302.03648*, 2023.

- [25] W. McKinney *et al.*, “Pandas: A foundational python library for data analysis and statistics,” *Python for high performance and scientific computing*, vol. 14, no. 9, pp. 1–9, 2011.
- [26] E. Belouadah, A. Popescu, and I. Kanellos, “A comprehensive study of class incremental learning algorithms for visual tasks,” *Neural Networks*, vol. 135, pp. 38–54, 2021.

APPENDIX

A. Knowledge Distillation Loss Leads to Failed Coverage

The loss function used in LWF can cause the model to fail to converge, as shown in figure A.1 and A.2. This results in the model being unable to classify the new task's classes. The issue arises because, in some cases, reducing the loss $\mathcal{F}'_t$ does not lead to correct label predictions. This occurs when the model accurately predicts the probabilities of the new classes, but the probability of the new classes $\mathcal{F}'_t$ is still lower than that of the previous classes $\mathcal{F}_{t-1}$ (which was not taken into consideration when calculating loss $\mathcal{F}'_t$). This phenomenon occurs specifically in the context of CIL when the fully connected (FC) layer is expanded to accommodate new tasks. It does not occur in the original Knowledge Distillation Loss paper, where the goal is to transfer knowledge from a large ensemble of models to a single model.

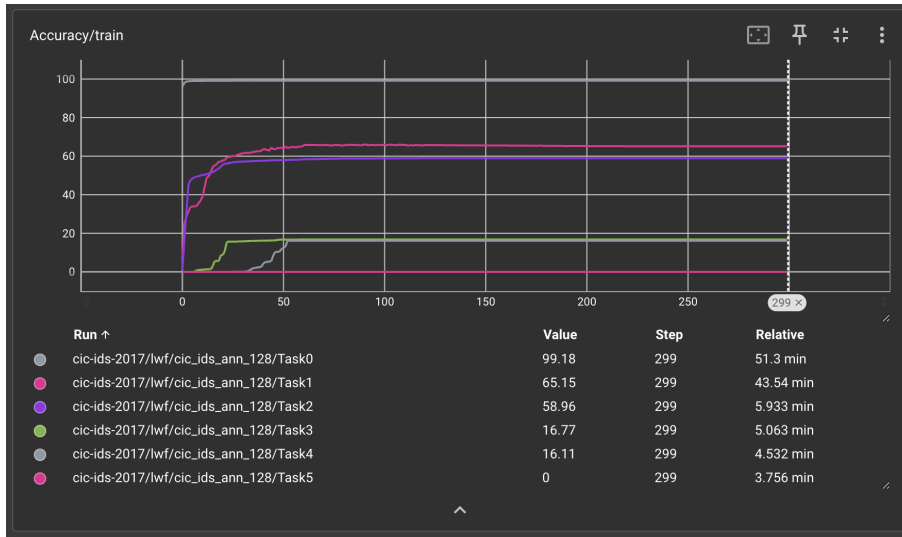


Figure A.1: Failed Coverage of LWF when trained on CIC-IDS-2017 dataset

```
import torch
from torch import tensor
from torch.nn import functional as F

# Train on class 4-5
outputs = tensor([[ -3.6737,  3.6739,  7.9670, -6.8142,  7.5660, -7.4853]])
# Predict on class 0-5
predicts = torch.max(outputs, dim=1)
print(predicts.indices) # Result is 2
target = tensor([4], dtype=torch.int64) # Target is 4
fake_target = target - 4 # Fake target is 0
loss = F.cross_entropy(outputs[:, 4:], fake_target)
print(round(loss.item(), ndigits=3)) # Loss is 0
```

Figure A.2: Pytorch Example of Failed Converge Training Instance